Q&A Bảo vệ ĐATN — PM QLKT

Tài liệu chuẩn bị bảo vệ ĐATN — PM QLKT

> Tài liệu trả lời các câu hỏi hay gặp khi bảo vệ đồ án về ứng dụng web. Mỗi câu hỏi có (1) câu trả lời ngắn 20–30 giây để đáp ngay, (2) phần chi tiết kỹ thuật để trả lời tiếp khi bị truy vấn sâu, (3) đoạn code thật trong project khi có liên quan, và (4) câu hỏi phản biện thường đi kèm.

Mục lục

- 0. Chiến thuật trả lời hội đồng
- A. Công nghệ và lý do chọn
- B. Kiến trúc và design pattern
- C. Bảo mật ứng dụng web
- D. Race condition và concurrency
- E. Logic chuỗi danh hiệu
- F. Cú pháp Prisma đối chiếu SQL
- G. Hiệu năng và mở rộng
- H. Kiểm thử
- I. Triển khai và vận hành
- J. Câu hỏi khoai và edge case
- K. Câu hỏi nghiệp vụ quân đội
- L. Khi không biết câu trả lời
- M. Khả năng bảo trì và mở rộng kiến trúc
- N. Tổng hợp chống tấn công và đánh giá an toàn
- O. Truy vấn nâng cao và tối ưu cơ sở dữ liệu

0. Chiến thuật trả lời hội đồng

Cấu trúc một lần trả lời chuẩn: 1. **Khẳng định mệnh đề chính** (1 câu): “Em xử lý X bằng cách Y.” 2. **Lý do chọn** (1 câu): “Em chọn Y vì Z (so với phương án W).” 3. **Bằng chứng cụ thể** (1 câu, tuỳ chọn): “Trong file `…`, đoạn em viết… cho ra kết quả… Em có viết test case `…` kiểm tra điều này.” 4. **Hạn chế còn tồn tại** (nửa câu): “Hiện tại em chưa làm được… nhưng đã đề xuất ở Chương 6.”

Nguyên tắc vàng: - Đừng nói “em không biết” trống không. Nói: “Em chưa kiểm chứng phần đó nhưng theo em hiểu thì… nếu sai mong thầy/cô chỉ giáo.” - Khi bí, kéo hội đồng về phần em làm tốt: “Để em lấy ví dụ cụ thể từ module X trong project…” - Đừng cãi quá 1 vòng. Hội đồng đúng → ghi nhận luôn: “Đúng

ạ, em sẽ ghi vào hướng phát triển.” - Câu hỏi mở rộng không có trong đồ án → trả lời “trên lý thuyết” rồi chốt “em có thể bổ sung sau khi có thời gian thử nghiệm.”

A. Công nghệ và lý do chọn

A.1 — Tại sao chọn Next.js 14 App Router thay vì Pages Router hay React thuần?

Ngắn: App Router cho phép Server Components giảm bundle JS phía client, và cấu trúc route lồng theo thư mục dễ tổ chức cho project có nhiều cấp menu như em.

Chi tiết: - Next.js 14 ổn định App Router từ tháng 10/2023, hỗ trợ tốt React Server Components — phần render trên server không cần gửi JS xuống client, giảm bundle ~30 % cho trang chỉ đọc dữ liệu (vd: trang danh sách quân nhân). - Layout lồng (`app/layout.tsx` → `app/admin/layout.tsx` → `app/admin/personnel/page.tsx`) giúp tách thanh điều hướng theo vai trò mà không cần higher-order component. - File-based routing tránh phải duy trì thủ công bảng route map. - So với React + Vite thuần: không có SSR/SSG sẵn → SEO (nếu sau này public ra Internet) sẽ kém hơn; phải tự config router. - So với Pages Router: API mới hơn (loading.tsx, error.tsx, parallel routes) thuận tiện cho UI phức tạp.

Hạn chế nói trước: “Em chưa khai thác hết Server Components — phần lớn trang vẫn là Client Component vì cần Ant Design và state.”

Phản biện thường gặp: “Tại sao không dùng Nuxt/Remix/SvelteKit?” → “Em đã quen React từ trước, nhân lực sau này tiếp nhận project cũng dễ tìm hơn so với Vue/Svelte.”

A.2 — Tại sao Express thay vì NestJS hoặc Fastify?

Ngắn: Express vẫn là framework Node.js có cộng đồng lớn nhất, em đã đủ kinh nghiệm áp đặt kiến trúc layered + Repository lên Express, không cần dependency-injection container của Nest cho quy mô project hiện tại.

Chi tiết: - Nest mạnh ở DI và decorator nhưng kéo theo TypeScript decorator + reflect-metadata + module system riêng — tốn thời gian học cho người maintain sau. - Fastify nhanh hơn ~30 % nhưng ecosystem plugin ít hơn (vd: `multer` tương đương `fastify-multipart` không hoàn toàn 1-1). - Em tự áp đặt convention `Route → Middleware → Controller → Service → Repository → Prisma` qua quy ước file đặt tên — đạt mục tiêu tách lớp mà không cần framework ép buộc.

Phản biện: “Sao không dùng Hono/Bun cho hiện đại hơn?” → “Bun chưa đạt 1.0 ở thời điểm em bắt đầu (đầu 2025), em ưu tiên ổn định hơn.”

A.3 — Tại sao Prisma thay vì TypeORM, Sequelize, hay raw SQL?

Ngắn: Prisma có schema-first (1 file `schema.prisma` là nguồn duy nhất), client tự sinh type-safe, migration tự sinh từ diff schema — giảm 80 % bug runtime do typo tên cột.

Chi tiết so sánh:

Tiêu chí	Prisma	TypeORM	Sequelize	Raw SQL (`pg`)
Type-safe	Tự sinh từ schema	Decorator + entity	Yếu	Không
Migration	Auto từ diff	Manual hoặc auto	Manual	Manual
Truy vấn lồng nhau	`include` / `select` rõ ràng	Lazy/eager phức tạp	Tương đối	Toàn quyền nhưng dài
Raw escape hatch	`$queryRaw` parameterized	`query()`	`query()`	Mặc định
Performance	Tốt, có connection pool	Tốt	Tương đương	Nhanh nhất
Học	Dễ nhất	Khó nhất (nhiều khái niệm)	Trung bình	Phụ thuộc người dùng

- Prisma sinh client tự động sau `npx prisma generate` → IDE autocomplete cho tên model, tên cột, kiểu dữ liệu.
- Migration `prisma migrate dev` tạo file SQL có thể commit, đảm bảo môi trường dev/staging/prod cùng schema.
- Hạn chế Prisma: query phức tạp dạng window function hoặc CTE phải fallback `$queryRaw` (em có sẵn vài chỗ trong dashboard service).

Phản biện: "Prisma có overhead không?" → "Có ~10–15 % cho query đơn giản so với raw `pg`, nhưng đổi lại type-safe và DX. Khi có endpoint chậm, em fallback `$queryRaw`."

A.4 — Tại sao PostgreSQL thay vì MySQL hoặc MongoDB?

Ngắn: PostgreSQL hỗ trợ JSONB, CTE, window function, partial index — phù hợp cho dữ liệu nửa-cấu-trúc như cờ thành tích trong `DanhHieuHangNam` và truy vấn phân tích dashboard.

Chi tiết: - Schema khen thưởng có 23 model có quan hệ chặt → cần quan hệ 1-N, M-N nguyên gốc → relational DB phù hợp hơn MongoDB. - MongoDB không có transaction multi-document mặc định trong môi trường standalone → khó dùng cho luồng phê duyệt cần ACID. - MySQL cũng tốt nhưng PostgreSQL có: - JSONB index (em dùng cho `co_quan_don_vi` trong `QuanNhan` và `payload` trong `SystemLog`). - `RETURNING` clause sau `INSERT/UPDATE/DELETE` (Prisma đã tự dùng). - `pg_dump` tích hợp sẵn cho sao lưu (em dùng trong `backup.service.ts`). - Foreign key `ON UPDATE CASCADE` chặt chẽ hơn MySQL.

Phản biện: "PostgreSQL có nặng cho LAN nội bộ?" → "Một instance Postgres ăn ~150 MB RAM ở idle, hoàn toàn chạy được trên server 4 GB như em đề xuất."

A.5 — Tại sao tách Joi ở backend và Zod ở frontend?

Ngắn: Joi tích hợp tốt với Express middleware có sẵn, Zod tích hợp tốt với React Hook Form và infer type cho TypeScript ở FE. Hai thư viện phục vụ hai môi trường khác nhau.

Chi tiết: - **Joi (BE):** dùng trong middleware `validate.ts`, mỗi endpoint có schema riêng (vd: `accountValidation.create`). Joi có `stripUnknown: true` để loại bỏ field thừa do client gửi (chống

mass assignment). - **Zod (FE):** infer kiểu trực tiếp từ schema → form data type-safe khi submit. Tích hợp `zodResolver` của React Hook Form rất ngắn. - Lý do **không dùng cùng 1 thư viện hai bên:** - Zod ở Express phải qua wrapper, không có ecosystem đầy đủ cho async validation. - Joi ở React phải tự viết resolver, infer type yếu. - **Hạn chế:** Schema bị duplicate ở 2 phía. Có thể dùng `tRPC` hoặc shared package nếu sau này monorepo.

Phản biện: "Sao không dùng class-validator như NestJS?" → "Em không dùng decorator để giữ tương thích với TS không có experimental flag."

A.6 — Tại sao JWT (access + refresh) chứ không phải session-based?

Ngắn: Server stateless dễ scale ngang khi sau này deploy nhiều instance; refresh token rotation cho phép thu hồi session từ server mà không cần Redis.

Chi tiết theo code thật (`auth.service.ts`): - **Access token:** 30 phút, ký bằng `JWT_SECRET`, payload chứa `id`, `username`, `role`, `quan_nhan_id`. - **Refresh token:** 7 ngày, ký bằng `JWT_REFRESH_SECRET` riêng, payload chỉ `id` và `username` để giảm bề mặt rò rỉ. - **Lưu trữ refresh:** lưu `refreshToken` trong cột `TaiKhoan.refreshToken` → mỗi tài khoản chỉ có 1 phiên hoạt động cùng lúc. Đăng nhập mới sẽ ghi đè và `emitToUser('force_logout')` cho session cũ. - **Verify access:** middleware `verifyToken` check JWT chữ ký + so sánh có refreshToken trong DB → nếu admin xoá refreshToken thì access token đang còn hạn cũng bị từ chối.

Yếu điểm trung thực: - JWT không thể thu hồi chỉ access token — em mitigate bằng cách check DB mỗi request (đã đánh đổi 1 query mỗi request lấy khả năng revoke). - Nếu access token bị lộ trong 30 phút → kẻ tấn công vẫn dùng được. Cách giảm: rút expire xuống 15 phút.

Token leak qua đâu — phòng thế nào: - **Log:** Express middleware chặn log header `Authorization` trong access log (winston format custom). - **Network:** LAN nội bộ → traffic không qua Internet → MITM khó. Khi nâng HTTPS với cert nội bộ, JWT trên đường dây được mã hoá TLS. - **Browser DevTools:** User vẫn thấy JWT trong Network tab — đây là rủi ro của mọi app dùng JWT. Mitigate: short expire + refresh rotation. - **localStorage XSS:** XSS đọc được localStorage. Em chống bằng React tự escape + không dùng eval — XSS gần như không thực hiện được.

Phản biện: "Lưu refreshToken trong DB là làm mất ưu điểm stateless?" → "Đúng, đây là đánh đổi giữa scaling và khả năng revoke. Em chọn revoke vì project chạy LAN, không cần scale ngang."

A.7 — Tại sao Socket.IO mà không phải WebSocket native hay SSE?

Ngắn: Socket.IO có fallback HTTP long-polling khi mạng nội bộ chặn WebSocket, room/namespace sẵn để gửi notification cho 1 user, và reconnect tự động.

Chi tiết: - WebSocket native không có fallback — nếu firewall LAN chặn, người dùng mất thông báo. - SSE chỉ 1 chiều server → client, không gửi ngược được. Em cần ngược lại cho ack. - Socket.IO room: `socket.join(userId)` → `io.to(userId).emit(...)` gửi đúng người. - Em dùng cho 3 luồng: - Notification đề xuất mới gửi cho ADMIN. - Force logout khi đăng nhập từ nơi khác. - Tiến trình import Excel lớn (push từng dòng).

Phản biện: "Sao không Pusher/Ably?" → "Phải gửi data ra Internet, vi phạm chính sách bảo mật LAN nội bộ."

A.8 — Ant Design + Tailwind CSS + shadcn/ui — tại sao 3 thư viện UI?

Ngắn: Ba thư viện phục vụ ba mục đích khác nhau, không trùng lặp: Ant Design cho component nghiệp vụ phức tạp (Form, Table, Modal), Tailwind cho spacing/layout, shadcn/ui cho component mở rộng cần tuỳ biến sâu.

Chi tiết: - **Ant Design:** form validation tích hợp, table có pagination/sort/filter sẵn, locale tiếng Việt — rút ngắn 50 % code so với tự build. - **Tailwind:** dùng cho layout grid, spacing margin/padding, responsive — không động đến component logic. - **shadcn/ui:** dùng cho 2–3 component đặc biệt (vd: command palette `Cmd+K`, advanced popover) mà Ant Design không có.

Hạn chế: Bundle CSS có thể overlap. Em đã purge Tailwind theo content và import từng component AntD theo nhu cầu (`import { Table } from 'antd'`).

Phản biện: "Có thể chỉ dùng Tailwind + Headless UI?" → "Có, nhưng phải tự xây Form, Table — tốn 4–6 tuần thêm."

A.9 — Tại sao Jest mà không phải Vitest hay Mocha?

Ngắn: Jest tích hợp `ts-jest` chạy file `.ts` không cần build, mocking sẵn, cộng đồng lớn nhất cho Node.js backend. Vitest mới hơn, tốt cho FE Vite nhưng chưa cần đổi.

Chi tiết: - 870 ca kiểm thử / 74 suite hiện chạy trong ~38 giây — chấp nhận được. - Jest snapshot testing chưa dùng nhiều, chủ yếu unit test pure function. - `jest --coverage` sinh báo cáo HTML tại `coverage/lcov-report/index.html`, đạt > 85 % cho `services/profile`, `services/eligibility`, `services/proposal`.

A.10 — ExcelJS, multer, bcrypt, nodemailer — vai trò?

Thư viện	Vai trò trong project	Tại sao chọn
ExcelJS	Đọc/ghi `.xlsx` cho import danh hiệu hằng năm và xuất danh sách	Xử lý formula, style, merged cell tốt hơn `xlsx` thuần
multer	Nhận file PDF quyết định và file Excel upload	Streaming, không nuốt RAM với file lớn
bcrypt	Hash password tài khoản	Adaptive cost (em dùng cost 10 = ~100 ms/hash)
nodemailer	(Hiện chưa kích hoạt) gửi email reset password	Có sẵn để bật khi cần

A.11 — Next.js 14 build sinh ra những file gì? Khác gì so với React + Vite?

Ngắn: Next.js 14 chia ra 3 nhóm artifact: file dev sinh khi `next dev`, file build sinh khi `next build`, file type sinh tự động cho TypeScript. React + Vite chỉ có nhóm dev và build, không có generation cho route file-based.

Khi gõ `next dev` lần đầu: - `next-env.d.ts` — file tự sinh ở root, khai báo type cho `*.module.css`, image import, env vars. **Đừng commit edit thủ công** — Next overwrite mỗi lần chạy. - `.next/cache/`

— cache webpack/SWC để dev start nhanh hơn lần 2. - `.next/types/` (Next 13.2+) — type cho route handler, link href, dynamic param. Bật bằng `experimental.typedRoutes` để type-check tên route.

Khi gõ `next build`: - `.next/server/` — code render trên server (RSC + route handler). - `.next/static/` — JS/CSS bundle gửi xuống browser, có content hash trong tên file. - `.next/standalone/` (nếu `output: 'standalone'` trong `next.config.js`) — bundle tự chứa Node modules, có thể `node server.js` chạy luôn. - `.next/build-manifest.json` + `app-build-manifest.json` — map route → JS chunk.

Em không có config `standalone` ở project này → deploy qua PM2 với `npm start` (alias `next start`). `.next/` toàn bộ phải được copy lên server.

Khác React + Vite thuần: | Việc | Next.js 14 | React + Vite | |—|—|—| | Routing | File-based (folder = route) | Phải tự setup `react-router` | | SSR/SSG/ISR | Tự động theo file `page.tsx` | Không có sẵn | | API endpoint | `app/api/.../route.ts` cùng repo | Phải BE riêng | | Image optimization | `next/image` resize on-demand | Phải tự xử lý | | Bundle analyzer | `@next/bundle-analyzer` | Phải tự setup | | Dev server | `next dev` (port 3000) | `vite` (port 5173) |

Em chọn Next.js dù không cần SSR vì: routing file-based + middleware tự động + hỗ trợ Server Components về sau khi cần tối ưu bundle.

Phản biện: "Em có dùng SSR/SSG không?" → "Hệ thống nội bộ cần đăng nhập, không có trang public → em dùng Client Components là chính. SSR chỉ giá trị khi có SEO public."

A.12 — App Router file conventions: page, layout, loading, error, … — kể chi tiết

Ngắn: Mỗi tên file đặc biệt trong `app/` có ngữ nghĩa cố định, Next.js tự render đúng vị trí. Em dùng 5 trong số đó cho project: `layout.tsx`, `page.tsx`, `error.tsx`, `not-found.tsx`, `loading.tsx`.

Bảng đầy đủ Next.js 14 hỗ trợ:

File	Vai trò	Project em có
`layout.tsx`	UI bao quanh nhiều page con, không re-render khi navigate giữa con	✓ `app/layout.tsx` (root) + `app/admin/layout.tsx` (sidebar admin)
`page.tsx`	UI cho route đó, làm route public	✓ tất cả route
`loading.tsx`	UI hiển thị khi page con đang fetch (Suspense boundary tự động)	Chưa dùng
`error.tsx`	Error boundary, catch lỗi từ page con	✓ `app/error.tsx`
`not-found.tsx`	Render khi `notFound()` được gọi hoặc route không match	✓ `app/not-found.tsx`
`template.tsx`	Như layout nhưng re-mount mỗi navigation	Chưa dùng
`route.ts`	Route handler (REST endpoint thay vì UI)	Chưa dùng — em có BE Express riêng
`default.tsx`	Fallback cho parallel route	Chưa dùng

Quy ước folder: - `app/(auth)/login/page.tsx` — group route, dấu `()` không vào URL → URL là `/login`. Em dùng để gom `login`, `change-password` mà không làm tăng path. - `app/admin/personnel/[id]/page.tsx` — dynamic param, `params.id` được Next inject vào prop của page. - `app/admin/personnel/[id]/edit/page.tsx` — nested route, URL là `/admin/personnel/abc123/edit`.

Render order:

```
RootLayout (app/layout.tsx)
  └ AdminLayout (app/admin/layout.tsx)
      └ ErrorBoundary (app/error.tsx)
          └ Suspense (loading.tsx nếu có)
              └ Page (app/admin/personnel/[id]/page.tsx)
```

Phản biện: "Em không dùng `loading.tsx`?" → "Em dùng `<LoadingState>` shared component bên trong page, kiểm soát chi tiết hơn — nhưng đúng là `loading.tsx` chuẩn Next hơn, em sẽ chuyển đổi nếu có thời gian."

A.13 — Server Components vs Client Components — em dùng cái nào?

Ngắn: Mặc định Next 14 App Router coi mọi component là Server Component. Em phải gắn `'use client'` ở đầu file để chuyển sang Client Component khi cần state/effect/event listener. Project em hầu hết là Client Component vì dùng Ant Design.

Server Component (mặc định): - Render trên server, kết quả là HTML + RSC payload (không gửi JS xuống). - KHÔNG dùng được: `useState`, `useEffect`, `onClick`, browser API (`window`,

`localStorage`). - Dùng được: async/await trực tiếp trong body, gọi DB/API ngay trong component. - Lợi ích: giảm bundle JS, fetch song song trên server, không leak secret xuống client.

Client Component (`'use client'`): - Render bootstrap trên server (HTML đầu) + hydrate trên browser. - Dùng được hooks, event handler, browser API. - Bắt buộc cho: form, modal, animation, AntD component (vì AntD dùng `useContext`).

Project em: - `'use client'` ở **đa số** page (do AntD `Form`, `Table`, `Modal` cần context). - Server Component **chỉ** dùng cho 2 layout root đơn giản (`app/layout.tsx`, `app/admin/layout.tsx`). - Trade-off: bundle to hơn nhưng DX (developer experience) đơn giản — không phải nhớ ranh giới.

Quy tắc thực dụng: - Component có `useState` / `useEffect` / `onClick` → `'use client'`. - Component import AntD/Tailwind plugin có hook → `'use client'`. - Component chỉ render markup tĩnh + fetch dữ liệu → có thể Server Component.

Hạn chế trung thực: "Em chưa khai thác hết Server Components — nếu chuyển 1 nửa số page sang RSC, bundle sẽ giảm thêm ~25 %."

Phản biện: "Component cha là Server, con là Client truyền props — props phải serializable?" → "Đúng. Em không truyền function/JSX qua ranh giới này, chỉ truyền data thuần."

A.14 — Tailwind + PostCSS + shadcn/ui — config file gì, hoạt động ra sao?

Ngắn: Tailwind sinh CSS theo class trong code (JIT). PostCSS là pipeline xử lý. shadcn/ui là CLI copy component vào repo, không phải npm package.

File cấu hình project em có: - `tailwind.config.ts` — khai báo `content: ['./src/**/*.{ts,tsx}']` để Tailwind scan class từ code, `theme.extend` thêm color palette tùy biến, `darkMode: 'class'` bật dark mode qua class trên `<html>`. - `postcss.config.js` — chạy `tailwindcss` + `autoprefixer` plugin. Next.js đọc file này tự động khi build. - `src/app/globals.css` — import 3 directive `@tailwind base/components/utilities`. File này được import 1 lần ở `app/layout.tsx`. - `components.json` — config shadcn/ui CLI: alias `@/components`, style `default`, base color `slate`. CLI dùng nó khi gõ `npx shadcn@latest add button`. - `src/lib/utils.ts` — chứa `cn()` helper (clsx + tailwind-merge), shadcn/ui dùng để merge class có conflict.

Tailwind hoạt động: 1. `next dev` → PostCSS chạy. 2. Tailwind plugin scan `content` glob, tìm class string (`text-red-500`, `flex`, ...) trong file `.tsx`. 3. Sinh CSS chỉ chứa class được dùng → bundle CSS final ~20-30 KB cho project em (so với 3 MB nếu include hết Tailwind).

shadcn/ui khác AntD: - AntD: import từ npm, version cố định, khó tùy biến sâu. - shadcn/ui: copy source code vào `src/components/ui/`, em sửa trực tiếp được. Dùng `kebab-case.tsx` là exception duy nhất trong project (tất cả component khác PascalCase).

Hạn chế: Bundle CSS có overlap nhẹ giữa AntD reset và Tailwind preflight. Em đã thử disable preflight (`corePlugins.preflight: false`) — không đáng kể.

Phản biện: "Sao không dùng styled-components hoặc emotion?" → "CSS-in-JS overhead runtime (~10-20 KB). Tailwind biên dịch lúc build, runtime cost = 0."

A.15 — TypeScript config: BE `strict: false`, FE strict — tại sao khác?

Ngắn: BE em đặt `strict: false` để giảm friction khi viết Joi validation và Prisma query lồng. FE bật strict đầy đủ vì component cần type-safe để IDE refactor an toàn.

File config: - `BE-QLKT/tsconfig.json`: `strict: false`, `strictNullChecks: false`, `target: ES2020`, `module: CommonJS`. Output không phải `.js` build (em dùng `tsx watch` ở dev và `tsc` ở production build vào `dist/`). - `FE-QLKT/tsconfig.json`: `strict: true`, `target: ES2017`, `module: esnext`, `moduleResolution: bundler`, `paths: { "@/*": ["./src/*"] }` cho path alias.

Lý do BE relax: - Joi schema trả `unknown`, ép cast nhiều chỗ — `strictNullChecks` ép thêm `if (x !== undefined)` rườm rà. - Prisma `findUnique` trả `T | null`, đôi khi em chắc chắn record tồn tại (vừa create xong) → cast `!` hoặc destructure không null check là acceptable. - BE đã có Joi validation ở route → input đã được làm sạch, runtime safety không phụ thuộc TS strict.

Lý do FE strict: - Component nhận props nhiều cấp lồng — strict null check cứu khỏi `Cannot read property 'x' of undefined` khi render. - Refactor tên field DB → IDE báo đỏ ngay nơi sai.

Trade-off em chấp nhận: BE có ~5 chỗ `as any` (đã loại hết trong commit gần đây), FE chỉ có 0 sau khi cleanup.

Phản biện: "Sao không bật strict cả 2?" → "Bật strict BE phải sửa ~80 chỗ liên quan đến Prisma null. Em ưu tiên tốc độ phát triển. Có thể bật dần qua flag `noUncheckedIndexedAccess` rồi mới đến full strict."

A.16 — Prisma CLI: `migrate dev` vs `db push` vs `generate` vs `migrate deploy` — khác gì?

Ngắn: Bốn lệnh phục vụ vòng đời khác nhau: `generate` sinh client TS, `migrate dev` tạo migration file ở dev, `db push` đồng bộ schema không tạo migration, `migrate deploy` áp migration đã có ở production.

Vòng đời em đang dùng:

Lệnh	Khi nào dùng	Hậu quả
`npx prisma generate`	Sau khi đổi `schema.prisma`, trước khi gõ code	Sinh `src/generated/prisma/` (em config custom output, không dùng `node_modules/.prisma` mặc định)
`npx prisma migrate dev`	Dev: thêm/sửa cột	Sinh file `prisma/migrations/<timestamp>_<name>/migration.sql` + auto chạy + auto `generate`
`npx prisma db push`	Dev: prototype nhanh, **không có data quan trọng**	Sync schema vào DB, **KHÔNG tạo migration file** — chỉ dùng nháp
`npx prisma migrate deploy`	Production: áp tất cả migration chưa chạy	Đọc folder `prisma/migrations/`, chạy theo thứ tự, không tương tác
`npx prisma migrate reset`	Dev: reset DB sạch	Drop + recreate + chạy lại tất cả migration + seed
`npx prisma studio`	Dev: GUI xem/edit data	Mở web UI port 5555

Điểm em đã trả giá học: - **Đổi tên cột có data → KHÔNG dùng `db push`** — `db push` sẽ DROP cột cũ + CREATE cột mới → mất hết data. Phải viết script `prisma.$executeRawUnsafe('ALTER TABLE x RENAME COLUMN old TO new')` trong `src/scripts/` trước, rồi mới `db push` để Prisma sync schema. - Em đã ghi rule này vào `BE-QLKT/CLAUDE.md` (AP-8) sau khi suýt mất data lần đầu.

Custom output: `schema.prisma` của em có:

```
generator client {
  provider = "prisma-client-js"
  output   = "../src/generated/prisma"
}
```

→ Client sinh vào `src/generated/prisma/` thay vì `node_modules/@prisma/client`. Lý do: control version đi kèm code, không phụ thuộc reinstall.

Phản biện: "Sinh client vào `src/generated/` thì có nên commit không?" → "Có, vì TS strict ở FE cần type, và CI nhanh hơn (không phải `prisma generate` lại). Trade-off là repo to hơn ~5 MB."

A.17 — ESLint, Prettier, husky/lint-staged — em setup thế nào?

Ngắn: FE có ESLint + Prettier, BE chỉ có Prettier. Cả hai bên không có pre-commit hook (`husky/lint-staged`) — em phải nhớ chạy `npm run format` thủ công.

FE config: - `.eslintrc.json` — ESLint legacy config, extends `next/core-web-vitals` + `prettier`, plugin `unused-imports` rule `no-unused-imports: error` để chặn import thừa. - `.prettierrc` — `semi: true, singleQuote: true, tabWidth: 2, printWidth: 100, trailingComma: 'es5', arrowParens: 'avoid'`. - Script: `npm run lint` (gọi `next lint`), `npm run format` (gọi `prettier --write`).

BE config: - `.prettierrc` cùng convention với FE. - KHÔNG có `.eslintrc` — em rely vào `tsc --noEmit` (`npm run typecheck`) và Prettier để giữ chuẩn. - Script: `npm run typecheck`, `npm run format`.

Hạn chế trung thực: - **Không có pre-commit hook** — nếu em quên chạy `format`, code messy có thể commit. Đây là tech debt em đã ghi vào `PROJECT_REVIEW.md` §LOW. - Có thể thêm `husky` + `lint-staged` chạy `prettier --write` + `tsc --noEmit` trên file staged để chặn commit lỗi format/type.

Phản biện: "Sao không dùng Biome thay ESLint + Prettier?" → "Biome mới (1.0 cuối 2023), em chưa migrate vì project đã ổn định. Để hướng phát triển."

A.18 — PM2 ecosystem + Nginx reverse proxy — config thế nào?

Ngắn: PM2 chạy BE Node.js (port 4000) và FE Next.js production server (port 3000) với auto-restart. Mỗi app có `ecosystem.config.js` riêng. Nginx đặt trước, terminate TLS, reverse proxy `/api/*` về BE và `/*` về FE.

`BE-QLKT/ecosystem.config.js` (rút gọn):

```
module.exports = {
  apps: [{
    name: 'be-qlkt',
    script: 'dist/index.js',         // compiled từ tsc, KHÔNG dùng tsx production
    instances: 1,
    exec_mode: 'fork',
    autorestart: true,
    max_memory_restart: '500M',      // restart nếu RSS > 500 MB (chống memory leak)
    env_file: '.env',
    log_date_format: 'YYYY-MM-DD HH:mm:ss',
    out_file: 'logs/out.log',
    error_file: 'logs/error.log',
  }],
};
```

`FE-QLKT/ecosystem.config.js`: tương tự, `script: 'node_modules/.bin/next'` + `args: 'start'`.

Khởi động: `pm2 start BE-QLKT/ecosystem.config.js && pm2 start FE-QLKT/ecosystem.config.js`. Lưu state: `pm2 save && pm2 startup` để tự khởi động lại sau reboot server.

Nginx (rút gọn):

```
server {
  listen 80;
  server_name qlkt.local;

  client_max_body_size 50M;  # cho upload Excel/PDF lớn

  location /api/ {
    proxy_pass http://localhost:4000;
    proxy_set_header Upgrade $http_upgrade;     # Socket.IO
    proxy_set_header Connection "upgrade";
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  }

  location /socket.io/ {
    proxy_pass http://localhost:4000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
  }

  location / {
    proxy_pass http://localhost:3000;
  }
}
```

Vai trò Nginx: - TLS termination (chứng chỉ self-signed cho LAN). - Static caching cho `/_next/static/*` (tăng tốc tải lần 2). - Buffer body upload — protect Node khỏi slowloris. - Single entry point → user chỉ thấy 1 origin, không gặp CORS.

Express phải `app.set('trust proxy', 1)` để `req.ip` lấy đúng IP client từ header `X-Forwarded-For`.

Phản biện: "Sao không dùng Caddy thay Nginx?" → "Caddy auto-HTTPS rất tiện cho public Internet, nhưng LAN nội bộ em đã có cert nội bộ, Nginx ổn định và phổ biến hơn ở Việt Nam."

A.19 — Logging, helmet, rate-limit, dotenv, cors — middleware Express còn lại

Ngắn: 5 middleware chuẩn cho Express production. Em dùng tất cả ngoại trừ logging file thì rely vào `console.error` + system_logs DB thay vì winston.

Thư viện	Mục đích	Config trong code
`helmet`	Set 14 security header (CSP, HSTS, X-Frame, …)	`app.use(helmet())` ở `index.ts`
`cors`	Whitelist origin	`app.use(cors({ origin: ALLOWED_ORIGINS, credentials: true }))`
`dotenv`	Load `.env` vào `process.env`	`import 'dotenv/config'` ở đầu `index.ts`
`express-rate-limit`	Chặn DoS / brute force	`authLimiter` 30 req/5min, `writeLimiter` 30 req/15min — file `configs/rateLimiter.ts`
`morgan`	Log HTTP request	**Em không dùng** — log qua `system_logs` DB cho audit, console output đủ ở dev

Vì sao không dùng `winston` riêng? - System log đã ghi vào DB (`system_logs` table) qua `writeSystemLog()` — có thể query/filter/visualize qua trang Admin. - PM2 đã tự ghi `logs/out.log` (stdout) và `logs/error.log` (stderr) — em chỉ cần `console.log` / `console.error` ở app code, PM2 capture lại. - Trade-off: PM2 không tự rotate file log → file có thể to dần. Cần thêm plugin `pm2-logrotate` (`pm2 install pm2-logrotate`) để rotate theo size hoặc thời gian. Đã ghi vào hướng phát triển.

Helmet tinh chỉnh (`src/index.ts:40-44`):

```
app.use(helmet({ crossOriginResourcePolicy: { policy: 'cross-origin' } }));
```

- Đặt `crossOriginResourcePolicy: cross-origin` để FE (port 3000) load được file PDF/Excel served từ BE (port 4000) — mặc định helmet là `same-origin`.
- Các header khác giữ default: HSTS (chỉ effect khi HTTPS), X-Frame-Options DENY, X-Content-Type-Options nosniff, X-XSS-Protection, …
- Đáng lẽ nên thêm `contentSecurityPolicy` nhưng AntD inject inline style → cần allow `'unsafe-inline'` cho `style-src`. Em chưa setup → ghi vào hướng phát triển.

Phản biện: “Sao không bật full CSP?” → “AntD chưa hỗ trợ nonce-based CSP. Khi nào AntD v6 ra (đã có roadmap), em sẽ migrate. Hiện LAN nội bộ rủi ro XSS thấp.”

A.20 — Thư viện FE phụ: dayjs, axios, chart.js, react-hook-form, react-pdf-viewer

Ngắn: 5 thư viện FE phụ trợ. Mỗi cái thay thế phương án “to” hơn để giữ bundle nhỏ.

Thư viện	Mục đích	Thay cho
`dayjs` (~7 KB)	Format/parse date, locale tiếng Việt	`moment.js` (~70 KB), date-fns (~13 KB tree-shakable)
`axios`	HTTP client với interceptor	`fetch` (phải tự wrap), TanStack Query (overkill cho CRUD đơn giản)
`chart.js` + `react-chartjs-2`	Biểu đồ dashboard	`recharts` (phình bundle), `apache echarts` (overkill)
`react-hook-form` + `@hookform/resolvers`	Form state + Zod validation	Formik (chậm hơn, mỗi keystroke re-render nhiều)
`@react-pdf-viewer/core`	Xem PDF quyết định inline	`iframe src=...` (không có UI điều khiển), `pdf.js` thuần (phải tự build UI)

Axios interceptor (`src/lib/axiosInstance.ts`): - Request: tự gắn `Authorization: Bearer <accessToken>` từ localStorage. - Response: nếu 401 → tự gọi `/api/auth/refresh` → retry request gốc 1 lần. Nếu refresh cũng 401 → redirect `/login`. - Lý do dùng axios thay fetch: interceptor pattern cleaner, retry logic ngắn hơn 50 % so với fetch wrapper.

dayjs locale:

```
import dayjs from 'dayjs';
import 'dayjs/locale/vi';
dayjs.locale('vi');  // → "tháng 5 năm 2026"
```

Phản biện: “Sao không dùng TanStack Query cho data fetching?” → “TanStack Query mạnh khi cần cache + invalidation phức tạp. Em chỉ có CRUD + form, custom hook `useFetch` / `useMutation` đủ — bundle gọn hơn ~30 KB.”

A.21 — Cơ chế 2 token (access + refresh) end-to-end — login → refresh → logout

Ngắn: Hai token có vai trò khác nhau: access token đóng vai trò “vé vào cửa” mỗi request (ngắn, 30 phút), refresh token đóng vai trò “thẻ thành viên” để xin vé mới (dài, 7 ngày). Mỗi lần refresh, server sinh CẢ access lẫn refresh mới (rotation) → token cũ vô hiệu hoá ngay.

4 giai đoạn:

1. Đăng nhập (`POST /api/auth/login`)

```
FE LoginForm                  BE auth.controller            BE auth.service
DB
     |                              |                              |
|
     |── { username, password } ──→ |                              |
|
     |                              |── login(input) ────────────→ |
|
     |                              |                              |──
findUnique(username) →|
     |                              |                              |←── account
──────────────|
     |                              |                              |
|
     |                              |                  bcrypt.compare(password, hash)
|
     |                              |                              |
|
     |                              |                  generateAccessToken({id,
username,     |
     |                              |                    role, quan_nhan_id}) —
JWT_SECRET    |
     |                              |                  generateRefreshToken({id,
username})   |
     |                              |                    — JWT_REFRESH_SECRET
|
     |                              |                              |
|
     |                              |                              |── update
refreshToken ──→|
     |                              |←── { accessToken,            |
|
     |                              |     refreshToken, user } ── |
|
     |                              |
|
     |←── 200 + tokens + user ─────|
|
     |
|
   localStorage.setItem('accessToken', ...)
|
   localStorage.setItem('refreshToken', ...)
   localStorage.setItem('role', 'username', 'userId', 'quan_nhan_id', 'ho_ten',
'don_vi_id')
```

File: `BE/src/services/auth.service.ts:35-95`, `FE/src/contexts/AuthContext.tsx:96-110`.

Vì sao lưu refresh trong DB? Ý tưởng "stateless" của JWT thuần KHÔNG cho phép thu hồi. Em đánh đổi: thêm 1 cột `TaiKhoan.refreshToken` để có khả năng force-logout. Khi user đăng nhập từ thiết bị khác → ghi đè cột này → thiết bị cũ không refresh được nữa → bị đẩy ra.

2. Request bình thường

```
FE axios interceptor        BE Express                  BE verifyToken middleware
DB
     |                           |                            |
|
   request.use((config) =>       |                            |
|
     config.headers.Authorization =                           |
|
     `Bearer ${localStorage.getItem('accessToken')}`)         |
|
     |                           |                            |
|
     |── GET /api/personnel ──────→|                            |
|
     |  Authorization: Bearer ... |── verifyToken(req,res,next)──→|
|
     |                           |                            |── jwt.verify(token,
|
     |                           |                            |    JWT_SECRET) →
payload    |
     |                           |                            |
|
     |                           |                            |── findUnique(id,
select:    |
     |                           |                            |    refreshToken)
──────────→|
     |                           |                            |←── { refreshToken:
'…' } ──|
     |                           |                            |
|
     |                           |                 if (!account.refreshToken) → 401
|
     |                           |                            |
|
     |                           |                 req.user = payload
|
     |                           |                 next() — vào controller
|
     |                           |                            |
|
     |←── 200 + data ────────────|                           |
|
```

File: `BE/src/middlewares/auth.ts:29-40`, `FE/src/lib/axiosInstance.ts:13-20`.

Tại sao check DB mỗi request? Để có thể revoke. Trade-off: 1 query thêm/request (~1ms với index trên `id`). Đáng cho LAN nội bộ.

3. Access token hết hạn → refresh (`POST /api/auth/refresh`) — có rotation

```
FE axios interceptor                  BE auth.service                    DB
     |                                     |                               |
   response.use(null, async (error) => {     |                               |
     if (status === 401 &&                   |                               |
         !originalRequest._retry &&          |                               |
         !isAuthRequest) {                   |                               |
                                             |                               |
       if (isRefreshing) {                   |                               |
         // Concurrent — đẩy vào failedQueue,|                               |
         // chờ refresh xong thì retry        |                               |
         return new Promise((resolve, reject) => {                           |
           failedQueue.push({ resolve, reject })                             |
         })                                  |                               |
       }                                     |                               |
                                             |                               |
       isRefreshing = true                   |                               |
       const stored = localStorage.getItem('refreshToken')                   |
                                             |                               |
   ────|── POST /api/auth/refresh ──────────→ |                               |
       |  { refreshToken: stored }           |                               |
                                             |── refreshAccessToken() ─────→ |
                                             |   jwt.verify(refreshToken,
|
                                             |     JWT_REFRESH_SECRET)
|
                                             |                               |──
findUnique(id) →|
                                             |                               |←──
account ──────|
                                             |
|
                                             |  if (account.refreshToken !==
refreshToken)      |
                                             |    throw 401  ← chống dùng refresh cũ
|
                                             |
|
                                             |  newAccessToken =
generateAccessToken(...)       |
                                             |  newRefreshToken =
generateRefreshToken(...)     |
                                             |                               |
|
                                             |                               |── update
|
                                             |                               |
refreshToken →  |
                                             |                               |
newRefreshToken |
       |←── 200 { accessToken: new,           |
|
       |         refreshToken: new } ────────|
|
                                             |
|
```

```
    localStorage.setItem('accessToken', new)
 |
    localStorage.setItem('refreshToken', new)
 |
        processQueue(null, new)               |
 |
        isRefreshing = false                  |
 |
        originalRequest.headers.Authorization = `Bearer ${new}`
 |
        return axiosInstance(originalRequest)  ← retry request gốc
 |
    }
```

File: `BE/src/services/auth.service.ts:111-138`, `FE/src/lib/axiosInstance.ts:58-180`.

Rotation: Mỗi lần refresh, BE sinh CẢ accessToken VÀ refreshToken mới, ghi đè cột DB. Refresh token cũ không còn match → reuse refresh cũ = 401 ngay. Đây là OWASP recommended pattern.

Concurrent refresh: Nếu 5 request 401 cùng lúc, chỉ 1 request đầu thực sự gọi `/auth/refresh`, 4 cái còn lại đứng `failedQueue` chờ → tránh sinh 5 cặp token.

4. Logout (`POST /api/auth/logout`)

```
FE                          BE auth.service                 DB
 |                                  |                            |
 |── POST /auth/logout ─────────────→|                            |
 |  { refreshToken }                |                            |
 |                                  |── updateMany({ refreshToken },|
 |                                  |   { refreshToken: null }) ──→|
 |                                  |                            |
 |←── 200 ──────────────────────────|                            |
 |
localStorage.clear()
router.push('/login')
```

File: `BE/src/services/auth.service.ts:145-152`.

Logout chỉ xoá refresh token trong DB, không invalidate access token đang còn 30 phút. Mitigate: middleware verifyToken check DB phát hiện `refreshToken: null` → 401 cho mọi request từ device đó NGAY LẬP TỨC.

Force logout cross-device: Đăng nhập máy mới → server ghi đè `TaiKhoan.refreshToken`. Máy cũ access token expire sau ≤30min → gọi refresh → DB.refreshToken không match → 401 → FE forceLogout. Em còn emit Socket.IO `force_logout` event để máy cũ logout NGAY thay vì chờ 30 phút.

Bảng tổng kết 2 token:

Tiêu chí	Access Token	Refresh Token
Secret	`JWT_SECRET`	`JWT_REFRESH_SECRET` (riêng)
TTL	30 phút	7 ngày
Payload	`{id, username, role, quan_nhan_id}`	`{id, username}` (tối giản)
Lưu trong DB	KHÔNG	CÓ (cột `TaiKhoan.refreshToken`)
Gửi mỗi request	CÓ (header Authorization)	KHÔNG (chỉ khi gọi refresh)
Rotation khi refresh	Sinh mới	Sinh mới + ghi đè DB
Có thể revoke server-side?	Gián tiếp (qua DB check)	Trực tiếp (set null)
Lưu phía client	localStorage	localStorage

Tại sao 2 token mà không phải 1?

Nếu chỉ 1 token TTL dài (vd 7 ngày): mỗi request gửi token này — nếu lộ qua log/MITM/XSS thì attacker có 7 ngày tự do → rủi ro lớn. TTL ngắn 30 phút thì user phải đăng nhập lại 48 lần/tuần → UX tệ.

Tách 2 token giải quyết: access ngắn 30 phút (lộ thì rủi ro chỉ 30 phút), refresh dài 7 ngày NHƯNG chỉ gửi qua HTTPS lúc gọi refresh, không lộ qua mỗi request. Best of both worlds.

Hạn chế trung thực: - Cả 2 token đều ở localStorage → vẫn dính XSS. Em dựa vào React auto-escape + không dùng `eval` / `dangerouslySetInnerHTML` chứa user input để hạn chế XSS. - Lý tưởng: refresh trong httpOnly cookie + CSRF token, access trong memory (không localStorage). Em chưa làm vì cần migrate cookie cross-origin (FE 3000 ↔ BE 4000 cần cấu hình `credentials: 'include'` + cookie SameSite). Ghi vào hướng phát triển.

Phản biện: - "Refresh rotation rồi attacker bắt được refresh cũ thì sao?" → "Refresh cũ không match DB → 401 ngay. Nhưng attacker có thể bắt được refresh MỚI vừa cấp, dùng tiếp. Detection: nếu thấy refresh cũ được gửi sau khi đã rotate → coi đó là suspicious, force logout cả phiên (em chưa implement, ghi vào hướng phát triển)." - "Vì sao không dùng `iat` / `jti` để revoke từng access token riêng lẻ?" → "Cần Redis blacklist để check `jti` mỗi request — phức tạp hơn cách hiện tại (1 cột DB). Trade-off, em chọn đơn giản." - "Vì sao không dùng PASETO thay JWT?" → "PASETO an toàn hơn (không có `alg: none` attack), nhưng ecosystem Node.js chưa phổ biến bằng. Em chọn JWT cho dễ tìm tài liệu."

A.22 — Cơ chế ký JWT chi tiết — HS256 vs RS256, tại sao 2 secret riêng?

Ngắn: Em dùng thuật toán HS256 (HMAC-SHA256) để ký JWT — symmetric, 1 secret cho cả ký lẫn verify. Hai loại token (access + refresh) ký bằng 2 secret KHÁC NHAU (`JWT_SECRET` và `JWT_REFRESH_SECRET`) để cô lập rủi ro: nếu 1 secret lộ thì chỉ 1 loại token bị compromise.

Cấu trúc JWT — 3 phần ngăn cách bởi dấu chấm:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 .
eyJpZCI6ImNseHl6IiwidXNlcm5hbWUiOiJhZG1pbiIsInJvbGUiOiJBRE1JTiIsInF1YW5fbmhhbl9pZCI6Im
FiYyIsImlhdCI6MTcxNDU2NzIzNCwiZXhwIjoxNzE0NTY5MDM0fQ .
X9k7JqRn8sV4tH6mP2cE3wY1bA5dF0gI
└────────────────── header ──────────────┘ └──────────────────────────────────── payload
──────────────────────────────────────────────────────┘ └──────── signature ────────┘
        base64url-encoded                                                    base64url-
encoded                                                  HMAC-
SHA256(header+'.'+payload, secret)

rồi base64url
```

Phần 1 — Header (mặc định khi `jwt.sign(payload, secret)`):

```
{ "alg": "HS256", "typ": "JWT" }
```

→ Báo cho phía verify biết dùng thuật toán gì để check chữ ký.

Phần 2 — Payload (claims) em đặt cho access token (`auth.service.ts:35-41`):

```
{
  "id": "clxyz123",
  "username": "admin",
  "role": "ADMIN",
  "quan_nhan_id": "abc456",
  "iat": 1714567234,        // issued at — jsonwebtoken tự thêm
  "exp": 1714569034         // expires at — tính từ expiresIn: '30m' → iat + 1800
}
```

Refresh token payload tối giản hơn: chỉ `{id, username, iat, exp(7d)}` — giảm bề mặt rò rỉ thông tin role.

Phần 3 — Signature (đây là phần ngăn user tự sửa payload):

```
signature = base64url(
  HMAC_SHA256(
    base64url(header) + '.' + base64url(payload),
    JWT_SECRET  // 256-bit secret, đọc từ .env
  )
)
```

Quá trình verify (BE middleware `auth.ts:29` gọi `jwt.verify(token, JWT_SECRET)`): 1. Split token bằng dấu `.` thành 3 phần. 2. Decode header để biết `alg` (BE em hardcode chấp nhận chỉ HS256, chống attack `alg: none`). 3. Tự tính lại `expectedSig = HMAC_SHA256(header+'.'+payload, JWT_SECRET)`. 4. So sánh `expectedSig` với `signature` gửi lên — KHÁC nhau dù 1 byte → throw `JsonWebTokenError`. 5. Nếu khớp → check `exp > now` — quá hạn → throw `TokenExpiredError`. 6. Nếu cả 2 OK → return decoded payload.

Vì sao user không tự sửa được role thành ADMIN? - User decode payload (base64url, không cần secret) → đổi `role: 'USER'` thành `role: 'ADMIN'` → re-encode → ghép lại signature CŨ. - BE

recompute signature từ header+payload mới (đã sửa) → ra signature MỚI khác signature cũ user gửi → throw error. - Để forge thành công, user phải biết `JWT_SECRET` để compute signature đúng — bất khả thi nếu secret 256-bit random và lưu trong `.env` chmod 600.

HS256 vs RS256 — tại sao em chọn HS256?

Tiêu chí	HS256 (em dùng)	RS256
Loại key	Symmetric — 1 secret	Asymmetric — private key (sign) + public key (verify)
Secret	256-bit random string	RSA keypair 2048+ bit
Tốc độ sign	~0.05 ms	~5 ms (chậm hơn ~100×)
Tốc độ verify	~0.05 ms	~0.2 ms
Phù hợp	Sign và verify ở cùng 1 service	Nhiều service verify, 1 service sign (microservices)
Phân phối khoá	Chỉ 1 nơi cần secret	Public key có thể chia sẻ rộng

Em chọn HS256 vì: - BE chỉ là 1 monolith Express duy nhất — vừa sign (login/refresh) vừa verify (mỗi request) → không cần tách private/public. - HS256 nhanh hơn ~100× lúc sign → login/refresh response nhanh hơn vài ms. - Setup đơn giản: 1 secret trong `.env`, không cần generate keypair.

Vì sao 2 secret riêng cho access và refresh? - **Defense in depth:** nếu `JWT_SECRET` lộ qua log/git accidentally → attacker forge access token, NHƯNG không forge được refresh token (cần `JWT_REFRESH_SECRET` khác). Khi user logout, refresh bị xoá DB → access forge cũng vô hiệu sau ≤30 phút (vì middleware check DB). - **Domain separation:** access dùng cho mọi request (bề mặt rộng), refresh chỉ dùng cho 1 endpoint `/auth/refresh` (bề mặt hẹp). Tách secret = nếu compromise xảy ra thì biết chính xác phải rotate secret nào. - **Code rõ ý:** đọc code thấy `JWT_SECRET` vs `JWT_REFRESH_SECRET` biết ngay token nào → tránh nhầm khi refactor.

Sinh secret an toàn:

```
openssl rand -base64 48        # → 64-char base64 string ~ 384 bit entropy
# hoặc:
node -e "console.log(require('crypto').randomBytes(48).toString('base64'))"
```

Em ghi vào `.env.example` hướng dẫn sinh secret, không commit value thật.

Hạn chế trung thực — secret rotation chưa làm: - Nếu `JWT_SECRET` lộ, em chỉ có thể đổi giá trị mới + restart server → MỌI user đang đăng nhập bị 401 ngay (access token cũ ký bằng secret cũ → verify fail) → phải đăng nhập lại. - Lý tưởng: dual-secret window — chấp nhận token ký bằng `JWT_SECRET_OLD` trong 30 phút sau khi rotate, sau đó chỉ chấp nhận `JWT_SECRET_NEW`. Em ghi vào hướng phát triển.

Phản biện thường gặp: - "JWT có vulnerable `alg: none` không?" → "`jsonwebtoken` library version 9+ mặc định KHÔNG chấp nhận `alg: none`. Em đang dùng v9.0.2." - "Secret 256-bit có đủ không?" → "Đủ. HMAC-SHA256 collision attack cần 2^128 phép tính → không khả thi với hardware hiện tại." - "Tại

sao không dùng EdDSA (Ed25519) thay HS256?” → “EdDSA mới hơn nhưng `jsonwebtoken` chưa support trực tiếp, phải dùng `jose` library — em chưa migrate.”

A.23 — Socket.IO luồng end-to-end — connect, auth, notification, reconnect, force logout

Ngắn: FE dùng Socket.IO client kết nối tới BE qua port 4000 ngay sau khi đăng nhập. Token JWT được gửi qua `auth` handshake để xác thực 1 lần → mỗi user join vào “phòng riêng” `user_<id>` → BE service emit notification vào phòng đó → chỉ người dùng đó nhận.

Stack thư viện: - BE: `socket.io` v4 (file `BE/src/utils/socketService.ts`) - FE: `socket.io-client` v4 (file `FE/src/hooks/useSocket.ts`) - Cả 2 nói chuyện qua giao thức Socket.IO (HTTP long-polling + WebSocket upgrade) trên cùng port 4000 với REST API.

Server bootstrap (`BE/src/index.ts`):

```
import { createServer } from 'http';
import { initSocket } from './utils/socketService';

const httpServer = createServer(app);          // Express app + Socket.IO chia chung
        HTTP server
initSocket(httpServer);                         // attach Socket.IO vào HTTP server
httpServer.listen(PORT);                        // listen 1 port duy nhất
```

→ Cùng 1 port 4000 phục vụ cả `GET /api/personnel` (HTTP) lẫn `wss://host:4000/socket.io/?EIO=4&...` (WebSocket).

Init Socket.IO server (`socketService.ts:25-67`):

```
io = new Server(httpServer, {
  cors: { origin: allowCorsOrigin, credentials: true },
  pingTimeout: 60000,        // sau 60s không nhận pong từ client → kill connection
  pingInterval: 25000,       // 25s gửi ping 1 lần để giữ kết nối sống
});

// Middleware xác thực — chạy 1 lần khi handshake, không phải mỗi event
io.use((socket, next) => {
  const token = socket.handshake.auth.token;       // lấy token từ FE gửi trong `auth:
        { token }`
  if (!token) return next(new Error('Không tìm thấy token'));
  try {
    const decoded = jwt.verify(token, JWT_SECRET); // CÙNG secret như middleware HTTP
    socket.user = decoded;                         // attach payload vào socket cho
        dùng sau
    next();
  } catch (error) {
    if (error instanceof jwt.TokenExpiredError) {
      return next(new Error('TOKEN_EXPIRED'));     // signal đặc biệt cho FE biết phải
        refresh
    }
    next(new Error('Token không hợp lệ'));
  }
});

io.on('connection', (socket) => {
  const userId = socket.user.id;
  socket.join(`user_${userId}`);                   // mỗi user 1 room riêng theo id
  socket.on('disconnect', () => socket.leave(`user_${userId}`));
});
```

Sequence diagram chi tiết:

Pha 1 — Connect và join room

```
FE useSocket hook              BE Socket.IO server                                BE
auth middleware
       |                                |
|
   io(SOCKET_URL, {                    |
|
     auth: { token: <accessToken> },   |
|
     transports: ['websocket','polling'],
|
     reconnection: true,               |
|
     reconnectionAttempts: 10,         |
|
   })                                  |
|
       |                               |
|
       |── HTTP GET /socket.io/?EIO=4&transport=polling&t=...
──────────────────────────────→|
       |   (long-polling handshake)    |
|
       |                               |── io.use((socket, next) => verify)
──────────────→|
       |                               |
|
       |                               |                      jwt.verify(token,
JWT_SECRET)
       |                               |                      → payload { id,
username, role, quan_nhan_id, iat, exp }
       |                               |                      socket.user = payload
       |                               |                      next() — pass
middleware
       |                               |
|
       |                               |── io.on('connection', socket => …)
──────────────|
       |                               |   socket.join(`user_${payload.id}`)
|
       |                               |
|
       |←── 200 OK + sid (session id) ──|
|
       |                               |
|
       |── Upgrade: websocket ──────────→|
|
       |←── 101 Switching Protocols ────|
|
       |                               |
|
   socket.on('connect', ...)           |
|
```

```
    updateStatus('connected')            |
  |
```

Pha 2 — Server emit notification → FE nhận

Ví dụ thực: Manager phê duyệt đề xuất → user người gửi nhận thông báo realtime.

```
BE proposal.service.approve()                BE socketService
FE useSocket hook
       |                                            |
|
   const notification = await                       |
|
     prisma.thongBao.create({ data: {               |
|
       nguoi_nhan_id: proposal.nguoi_de_xuat_id,    |
|
       loai: 'PROPOSAL_APPROVED',                   |
|
       tieu_de: 'Đề xuất được duyệt',               |
|
       noi_dung: '...'                              |
|
     }})                                            |
|
       |                                            |
|
       |── emitNotificationToUser(                  |
|
       |     proposal.nguoi_de_xuat_id,             |
|
       |     notification) ────────────────────────→|
|
       |                                            |
|
       |                                io.to(`user_${userId}`).emit(
|
       |                                  'new_notification', notification)
|
       |                                            |
|
       |                                            |── WebSocket frame
──────────────────────→|
       |                                            |   { event: 'new_notification',
|
       |                                            |     data: { id, tieu_de, ... } }
|
       |                                            |
|
       |                                            |
socket.on('new_notification', n => onNotificationRef.current(n))
       |                                            |
|
       |                                            |              → React state
setNotifications(prev => [n, ...prev])
       |                                            |              → AntD
message.info(n.tieu_de) toast
       |                                            |              → badge số chưa đọc
tăng 1
```

File: `BE/src/helpers/notification/helpers.ts:94, 104`, `BE/src/utils/socketService.ts:73-76` (`emitNotificationToUser`).

Vì sao dùng room `user_<id>` thay vì broadcast? - Mỗi notification có người nhận cụ thể (`nguoi_nhan_id`). Broadcast → mọi user thấy → leak data. - Room cho phép gửi targeted: `io.to('user_X').emit(...)` chỉ tới socket trong room đó. Nếu user X có 2 tab mở (cùng login) → cả 2 tab đều nhận (vì 2 socket cùng join `user_X`).

Pha 3 — Access token hết hạn giữa kết nối Socket.IO

Đây là edge case quan trọng: socket đang connected, token expire (sau 30 phút) → server reject reconnect.

```
FE socket                        BE Socket.IO middleware                FE useSocket
   |                                    |                                     |
   (kết nối sống đã 30 phút)            |                                     |
   ping/pong vẫn duy trì                 |                                     |
                                        |                                     |
   (mạng rớt 1 giây — laptop sleep)     |                                     |
   socket disconnected                   |                                     |
                                        |                                     |
   reconnection auto kick in            |                                     |
   io reconnect with same auth.token    |                                     |
   |── handshake với token cũ ─────────→|                                     |
                                        |── jwt.verify → TokenExpiredError ──→|
                                        |← throw new Error('TOKEN_EXPIRED') ─|
                                        |                                     |
   socket.on('connect_error', err) ←─── err.message === 'TOKEN_EXPIRED'    |
   if (err.message === 'TOKEN_EXPIRED') {                                    |
     const stored = localStorage.getItem('refreshToken')                     |
     const res = await axios.post('/api/auth/refresh', { refreshToken: stored })
     const newToken = res.data.data.accessToken                              |
     localStorage.setItem('accessToken', newToken)                           |
     socket.auth.token = newToken                                            |
     window.dispatchEvent(new CustomEvent('tokenRefreshed',                  |
       { detail: { accessToken: newToken } }))    ← sync với axios interceptor
     socket.connect()  ← retry handshake với token mới                       |
   }
   |── handshake với token mới ──────→|
                                        |── jwt.verify OK → next() ──────────→|
   socket.on('connect') ←─── reconnected                                     |
```

File: `FE/src/hooks/useSocket.ts:65-90`.

Custom event `tokenRefreshed` đáng chú ý: Khi axios interceptor (REST flow) refresh token, nó dispatch event này → useSocket lắng nghe → update `socket.auth.token` luôn → khỏi phải refresh 2 lần (1 bởi axios, 1 bởi socket).

Pha 4 — Force logout cross-device

Tình huống: User đăng nhập máy A → đăng nhập máy B → máy A phải bị đẩy ra ngay.

```
Máy B login                     BE auth.service                       DB
Máy A
     |                                |                                  |
|
   POST /api/auth/login                                                  |
|
     |── { username, password } ────→|                                 |
|
     |                                |── login() ──────────────────────
|
     |                                |   newRefreshToken = ...           |
|
     |                                |── findFirstActiveSocket(user A's id)
|
     |                                |     từ Socket.IO io.sockets.adapter.rooms
────────────|
     |                                |                                  |
|
     |                                |── emitToUser(userId,             |
|
     |                                |     'force_logout',              |
|
     |                                |     { message: 'Tài khoản vừa đăng nhập từ
thiết bị khác' })
     |                                |                                  |
|
     |                                |                                  |── WS frame
─────────→|
     |                                |                                  |   { event:
'force_logout', data: {…} }
     |                                |                                  |
|
     |                                |                                  |
socket.on('force_logout', data => {
     |                                |                                  |
onForceLogoutRef.current?.(data)
     |                                |                                  |      })
     |                                |                                  |      →
AuthContext.logout()
     |                                |                                  |      →
localStorage.clear()
     |                                |                                  |      →
router.push('/login')
     |                                |                                  |      → AntD
modal "Tài khoản đã đăng nhập ở nơi khác"
     |                                |                                  |
|
     |                                |── update TaiKhoan.refreshToken =
newRefreshToken (B's) |
     |                                |                                  |
|
     |←── 200 + tokens ──────────────|                                 |
|
```

File: `BE/src/services/auth.service.ts` (login), `FE/src/hooks/useSocket.ts:90` (`socket.on('force_logout')`).

Ngay cả khi máy A offline lúc B login (không nhận được force_logout): khi máy A online lại + access token expire + gọi refresh → BE thấy `account.refreshToken !== request.refreshToken` (vì đã bị B ghi đè) → 401 → axios interceptor force logout → cùng kết quả.

Pha 5 — Disconnect

```
FE đóng tab / điều hướng           BE
     |                                |
   browser fires unload              |
   socket emits disconnect           |
   useSocket cleanup function:       |
     socket.disconnect()             |
     |── close frame ──────────────→|
                                     |── io.on('disconnect', ...) ──→
                                     |   socket.leave(`user_${userId}`)  ← tự động khi
socket close
```

Giao thức transport — vì sao có cả `websocket` và `polling`?

Socket.IO mặc định thử upgrade lên WebSocket sau khi handshake bằng HTTP polling. FE em config `transports: ['websocket', 'polling']` — thử WebSocket trước, fallback polling nếu LAN/firewall chặn.

Cấu hình reconnection (`useSocket.ts:48-54`):

```
{
  reconnection: true,
  reconnectionDelay: 1000,            // delay đầu tiên 1s
  reconnectionDelayMax: 5000,         // backoff tối đa 5s
  reconnectionAttempts: 10,           // bỏ cuộc sau 10 lần thử
}
```

→ Wifi rớt 30 giây? Socket.IO tự reconnect 10 lần với delay 1s, 1s, 2s, 4s, 5s, 5s, … → tổng ~35 giây. Sau đó FE hiển thị badge "Disconnected" — user vẫn dùng REST được (chỉ mất realtime).

Hạn chế trung thực: - Em chưa scale ngang Socket.IO. Nếu deploy 2 instance BE, room `user_X` ở instance 1 không thấy được user emit từ instance 2. Cần `socket.io-redis-adapter` để pub/sub event qua Redis. Đã ghi vào hướng phát triển. - Polling fallback nuốt nhiều CPU/memory hơn WebSocket. Trên LAN nội bộ Học viện, WebSocket luôn work → fallback hiếm khi kích hoạt.

Phản biện: - "Sao không lưu auth token vào cookie httpOnly thay handshake `auth: {token}`?" → "Cookie tự gửi mỗi request, an toàn hơn localStorage, nhưng cross-origin (FE 3000 ↔ BE 4000) cần config phức tạp + CSRF token. Em chọn handshake auth cho đơn giản, chấp nhận rủi ro XSS." - "Vì sao verify token ở handshake mà không verify lại mỗi event?" → "Verify mỗi event tốn ~0.5 ms × hàng nghìn event/s → quá tốn. Token chỉ verify lúc connect; nếu user bị revoke giữa session → BE đóng connection bằng `socket.disconnect()` từ event handler `force_logout` flow." - "Có rate-limit Socket.IO event không?" → "Chưa. Socket.IO có thư viện `socket.io-rate-limiter` nhưng em chưa setup vì user nội bộ. Đã ghi hướng phát triển."

B. Kiến trúc và design pattern

B.1 — Mô tả kiến trúc tổng thể trong 1 phút

Trả lời mẫu: “Hệ thống chia thành ba tầng. Tầng frontend là Next.js 14 App Router chạy trên cổng 3000. Tầng backend là Express + TypeScript chạy trên cổng 4000, được tổ chức theo 6 lớp: Route → Middleware → Controller → Service → Repository → Prisma. Tầng dữ liệu là PostgreSQL 15. Hai bên FE và BE giao tiếp qua REST API và Socket.IO cho thông báo thời gian thực. Toàn bộ vận hành trên LAN của Học viện qua PM2 và Nginx reverse proxy.”

B.2 — Tại sao layered architecture mà không phải MVC pure?

Ngắn: MVC truyền thống ghép chặt View với Controller. Project em có FE riêng (Next.js), BE thuần là REST API → MVC mất ý nghĩa. Layered cho phép tách rõ trách nhiệm BE.

Chi tiết: - Route chỉ định nghĩa HTTP path + chain middleware — không có logic. - Middleware làm cross-cutting concern: auth, validate, audit log, unit filter. - Controller chỉ parse request → gọi service → format response qua `ResponseHelper`. Body controller không quá 15 dòng (anti-pattern AP-2). - Service chứa business logic; được chia nhỏ thành sub-folder khi > 800 LOC (vd: `services/proposal/approve/`). - Repository wrap Prisma, expose method ngữ nghĩa (`accountRepository.findUniqueRaw`) thay vì `prisma.taiKhoan.findUnique` rải khắp. - Prisma là tầng cuối, không bị gọi trực tiếp từ Controller (anti-pattern AP-1).

B.3 — Lợi ích của Repository Layer? Có overengineering không?

Ngắn: Repository giúp Service không phụ thuộc vào Prisma cụ thể, dễ thay ORM trong tương lai và dễ mock trong test.

Chi tiết: - Trước khi có Repository (commit `9bd12f6`), Service gọi `prisma.quanNhan.findMany(...)` trực tiếp. Test phải mock `prisma` global → fragile. - Sau Repository: Service gọi `quanNhanRepository.findActiveInUnit(unitId)`. Test mock `quanNhanRepository.findActiveInUnit` đơn giản hơn. - **Có overengineering không?** Em thừa nhận với endpoint CRUD đơn giản, Repository chỉ là proxy mỏng. Nhưng với module phức tạp (proposal, profile, eligibility) thì lợi ích rõ.

Phản biện: “Sao không dùng Active Record như Sequelize?” → “Active Record gắn data và behavior chung — service layer mất tác dụng. Em chọn anemic model.”

B.4 — Strategy Pattern cho 7 loại đề xuất — kể chi tiết

Ngắn: Mỗi loại đề xuất có 1 class implement interface `ProposalStrategy` với 4 method chuẩn (`buildSubmitPayload`, `validateApprove`, `importInTransaction`, `buildSuccessMessage`). Một REGISTRY map enum loại → instance. Caller dispatch qua `getStrategy(type).method(...)`.

Chi tiết code:

```ts
// services/proposal/strategies/proposalStrategy.ts
export interface ProposalStrategy {
  buildSubmitPayload(input: SubmitInput): Promise<SubmitPayload>;
  validateApprove(ctx: ApproveContext): Promise<void>;
  importInTransaction(tx: Prisma.TransactionClient, items: ImportItem[]):
        Promise<void>;
  buildSuccessMessage(result: ImportResult): string;
}

// services/proposal/strategies/index.ts
const REGISTRY: Record<ProposalType, ProposalStrategy> = {
  CA_NHAN_HANG_NAM: caNhanHangNamStrategy,
  DON_VI_HANG_NAM: donViHangNamStrategy,
  NIEN_HAN: nienHanStrategy,
  HC_QKQT: hcQkqtStrategy,
  KNC: kncStrategy,
  CONG_HIEN: congHienStrategy,
  NCKH: nckhStrategy,
};

export function getStrategy(type: ProposalType): ProposalStrategy {
  const strategy = REGISTRY[type];
  if (!strategy) throw new Error(`Không có strategy cho loại đề xuất ${type}`);
  return strategy;
}
```

Lợi ích so với `if/else` 7 nhánh: - Thêm loại mới: tạo 1 file `<type>Strategy.ts` + thêm 1 dòng vào REGISTRY. Không động đến `if/else` chính. - Test 1 strategy không kéo theo 6 strategy khác. - Hai strategy “single medal” (HC_QKQT, KNC) chia sẻ qua helper `singleMedalImporter` → DRY mà vẫn rõ ràng.

Phản biện: “Sao không dùng class abstract?” → “Interface đủ ràng buộc; class abstract sẽ kéo theo state mà strategy không cần.”

B.5 — Khi nào tách module? Tiêu chí cụ thể?

Ngắn: File > 500 LOC xem xét tách. File > 800 LOC bắt buộc tách concern. File > 1000 LOC như `approve.ts` (2001 LOC trước refactor) phải tách thành `<feature>.ts` orchestration mỏng + sub-folder cho từng concern.

Pattern áp dụng:

```
services/proposal/
├── approve.ts                    # < 500 LOC, public API + flow chính
└── approve/
    ├── types.ts                  # Shared interfaces
    ├── validation.ts             # Pre-flight checks
    ├── decisionMappings.ts       # Decision metadata + PDF persist
    └── import.ts                 # Transactional import dispatch
```

B.6 — Anti-pattern em đã chủ động tránh

Em viết sẵn 9 anti-pattern trong `BE-QLKT/CLAUDE.md` từ AP-1 đến AP-9: - AP-1: Controller gọi Prisma trực tiếp. - AP-2: Controller chứa business logic > 15 dòng. - AP-3: Helper gọi DB hoặc service (helper phải pure). - AP-4: Duplicate logic ở nhiều service không extract. - AP-5: Service gọi service vòng tròn hoặc chuỗi quá 3 cấp. - AP-6: Hardcoded role/status/danh hiệu thay vì import từ `constants/`. - AP-7: Response không qua `ResponseHelper`. - AP-8: `prisma db push` cho cột có data → mất data. - AP-9: Catch error rồi đẩy detail kỹ thuật vào message cho user.

C. Bảo mật ứng dụng web

C.1 — IDOR (Insecure Direct Object Reference): nếu user đoán URL `/api/personnel/123` của người khác?

Đây là câu hỏi rất hay bị hỏi. Trả lời chuẩn:

Ngắn: Em chống IDOR bằng 3 lớp: (1) middleware `verifyToken` chặn request không có JWT, (2) `requireRole` chặn vai trò thấp truy cập endpoint không được phép, (3) trong service em check ownership: nếu USER thì chỉ trả dữ liệu khớp `req.user.quan_nhan_id`; nếu MANAGER thì lọc theo cây đơn vị qua `unitFilter`.

Chi tiết các trường hợp:

Tình huống	Ai gọi	Có chặn được không	Cơ chế
`GET /api/personnel/<other_id>`	USER	Có — service kiểm `if (user.role === USER && id !== user.quan_nhan_id) throw ForbiddenError`	Ownership check trong service
`GET /api/personnel/<id_outside_unit>`	MANAGER	Có — `unitFilter` middleware tính cây đơn vị từ `req.user.quan_nhan_id`, sau đó service `WHERE id IN (personnelIds)`	Unit filter
`POST /api/proposals` (tạo đề xuất)	USER	Có — `requireManager` reject 403 trước khi vào controller	Role middleware
`DELETE /api/personnel/<id>`	MANAGER	Có — `requireAdmin` reject	Role middleware
`GET /api/profiles/annual/<other_id>`	USER	Có — service check `id !== req.user.quan_nhan_id`	Ownership

Code thật từ `middlewares/auth.ts`:

```
const checkRole = (allowedRoles: string[]) => {
  return (req: Request, res: Response, next: NextFunction): void => {
    if (!req.user) {
      res.status(401).json({ success: false, message: 'Vui lòng đăng nhập trước.' });
      return;
    }
    if (!allowedRoles.includes(req.user.role)) {
      res.status(403).json({
        success: false,
        message: `Chỉ ${allowedRoles.join(', ')} mới có quyền thực hiện thao tác
        này.`,
      });
      return;
    }
    next();
  };
};

const requireSuperAdmin = checkRole([ROLES.SUPER_ADMIN]);
const requireAdmin = checkRole([ROLES.SUPER_ADMIN, ROLES.ADMIN]);
const requireManager = checkRole([ROLES.SUPER_ADMIN, ROLES.ADMIN, ROLES.MANAGER]);
```

Code thật từ `middlewares/unitFilter.ts`:

```
const attachUnitFilter = async (req, res, next) => {
  if (req.user?.role !== ROLES.MANAGER) {
    req.unitFilter = null;
    return next();
  }
  const quanNhanId = req.user?.quan_nhan_id;
  // Lấy đơn vị TỪ DB của user, KHÔNG lấy từ query → không bị giả mạo
  const unitInfo = await getUnitInfo(quanNhanId);
  req.unitFilter = unitInfo;
  next();
};
```

Phản biện thường gặp: - "Nếu MANAGER tự đổi `req.unitFilter` được không?" → "Không. Middleware tính từ `req.user.quan_nhan_id`, mà `req.user` được decode từ JWT chữ ký bằng `JWT_SECRET`. Sửa JWT phải biết secret server." - "Tự sửa JWT bằng cách đổi `role` thành ADMIN?" → "JWT có chữ ký HMAC; sửa payload không update chữ ký → `jwt.verify` fail."

C.2 — SQL Injection: làm sao chống?

Ngắn: Prisma tự parameterize mọi truy vấn — `prisma.user.findMany({ where: { name: input } })` sinh `SELECT ... WHERE name = $1` chứ không nối chuỗi. Chỗ duy nhất em dùng raw là `$queryRaw` template literal — vẫn parameterize tự động vì là tagged template.

Chi tiết:

```
// AN TOÀN — Prisma parameterize tự động
await prisma.quanNhan.findMany({ where: { ho_ten: { contains: userInput } } });

// AN TOÀN — $queryRaw tagged template parameterize $1, $2
const userId = '...';
await prisma.$queryRaw`SELECT * FROM "QuanNhan" WHERE id = ${userId}`;

// NGUY HIỂM — $queryRawUnsafe nối chuỗi
await prisma.$queryRawUnsafe(`SELECT * FROM "QuanNhan" WHERE id = '${userId}'`);
```

Em chỉ dùng `$queryRawUnsafe` ở đúng 1 chỗ trong `scripts/renameColumn.ts` — chạy local một lần để rename column, không nhận input từ user.

Phản biện: "Validate input có cần thiết khi đã có Prisma?" → "Có, vì validation còn để chặn business rule (vd: năm phải 1900-2100), không chỉ chống SQLi."

C.3 — XSS (Cross-Site Scripting)

Ngắn: React tự escape mọi text khi render qua `{value}` → script tag bị render thành text. Em không dùng `dangerouslySetInnerHTML` ở bất cứ đâu trừ chỗ render PDF preview (đã sanitize).

Chi tiết: - **Stored XSS:** ai đó nhập `<script>alert(1)</script>` vào trường ghi chú → React render thành text literal, không execute. - **Reflected XSS:** error message từ server dạng "Không tìm thấy `<input>`" → React escape khi render trong `<Alert>`. - **DOM-based XSS:** em không có chỗ nào `eval()`, `innerHTML =`, hay `new Function()` từ user input.

Header bảo vệ thêm: em đặt `helmet()` trong `app.ts` để set `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Content-Security-Policy` mặc định.

C.4 — CSRF (Cross-Site Request Forgery)

Ngắn: Em dùng JWT trong header `Authorization: Bearer <token>` thay vì cookie. Browser không tự gửi header `Authorization` cho cross-origin request → CSRF không lợi dụng được.

Chi tiết: - Nếu lưu JWT trong localStorage: kẻ tấn công không đọc được vì same-origin policy. - Nếu lưu trong cookie HttpOnly: phải bật `SameSite=Strict` để chống CSRF. - Em chọn header → tránh hoàn toàn CSRF.

Trade-off: XSS sẽ đọc được localStorage. Em mitigate bằng React tự escape + không dùng eval — XSS gần như không thực hiện được.

C.5 — Brute force password

Ngắn: `authLimiter` chặn 10 request / 15 phút / IP cho `/api/auth/login`. Sau 10 lần sai, IP đó bị reject 429 Too Many Requests trong 15 phút.

Code thật:

```
// configs/rateLimiter.ts
export const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 10,
  message: { success: false, message: 'Quá nhiều yêu cầu, vui lòng thử lại sau 15
        phút' },
});
```

Phản biện: "Kẻ tấn công đổi IP qua proxy thì sao?" → "Đúng, em mới chống brute đơn giản. Nâng cấp sẽ có account lockout sau N lần sai liên tiếp."

C.6 — Mật khẩu lưu trữ thế nào?

Ngắn: Hash bằng `bcrypt` cost 10 (~100 ms/hash). Không lưu plaintext. Không log password vào audit log.

Cost = 10 nghĩa là gì: 2^{10} = 1024 vòng key derivation. Mỗi hash mất ~100 ms trên CPU thông thường.

So sánh chi phí vs an toàn:

Cost	Thời gian / hash	Đánh giá
8	~25 ms	Yếu — attacker brute ~1000 password/giây
10	**~100 ms**	**Acceptable — em đang dùng**
12	~300 ms	Strong nhưng login chậm
14	~1 s	Quá chậm cho UX

Tự động tăng cost theo thời gian: Mỗi vài năm tăng cost 1 đơn vị khi CPU mạnh hơn. Hash cũ vẫn verify được vì cost được embedded trong hash (`$2b$10$...`).

Phản biện: "Sao không Argon2 mạnh hơn?" → "Argon2 mạnh hơn nhưng bcrypt đủ cho ngữ cảnh LAN nội bộ. Đổi bcrypt → Argon2 sau dễ — chỉ thay 1 helper."

C.7 — File upload: chống upload file độc hại

Ngắn: `multer` config giới hạn (1) MIME type whitelist qua helper `createFileFilter`, (2) size max 10 MB cho phần lớn (50 MB cho ad-hoc award), (3) tách 2 storage strategy: `memoryStorage` cho file xử lý ngắn (parse Excel), `diskStorage` cho file cần lưu lâu dài (PDF quyết định) — lưu vào thư mục riêng ngoài web root để không bị execute như script.

Code thật (`configs/multer.ts`):

```
const createFileFilter = (allowedMimes: string[], errorMsg: string) =>
  (req, file, cb: FileFilterCallback) => {
    if (allowedMimes.includes(file.mimetype)) cb(null, true);
    else cb(new Error(errorMsg));
  };

// Excel parse — memoryStorage vì xử lý xong là drop
export const excelUpload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 10 * MB },
  fileFilter: createFileFilter([MIME.XLSX, MIME.XLS], 'Chi chap nhan file Excel'),
});

// PDF quyết định — diskStorage vì cần lưu để download sau
export const pdfDecisionUpload = multer({
  storage: multer.diskStorage({ destination: './uploads/decisions', filename: ... }),
  limits: { fileSize: 10 * MB },
  fileFilter: createFileFilter([MIME.PDF], 'Chi chap nhan file PDF'),
});
```

Dùng helper `createFileFilter` thay vì inline → không lặp logic ở 6 multer config khác nhau (DRY).

Phản biện: "MIME type có thể bị giả?" → "Đúng. Em check thêm magic byte 4 byte đầu cho PDF (`%PDF-`) ở service. Chưa check magic byte cho .xlsx — sẽ bổ sung."

C.8 — Path traversal trên API tải file PDF

Ngắn: Endpoint `GET /api/decisions/download/:soQuyetDinh` lookup `FileQuyetDinh` theo `so_quyet_dinh` lấy `file_path` từ DB → không bao giờ nhận `file_path` trực tiếp từ user.

```
// services/decision.service.ts
const decision = await fileQuyetDinhRepository.findBySoQuyetDinh(soQuyetDinh);
if (!decision?.file_path) throw new NotFoundError('File quyết định không tồn tại');
const safePath = path.join(__dirname, '../../uploads',
        path.basename(decision.file_path));
res.download(safePath);
```

`path.basename` strip mọi `../` → không leak file ngoài thư mục `uploads/`.

C.9 — Privilege escalation: USER tự tăng role thành ADMIN

Ngắn: Role nằm trong JWT chữ ký, chỉ server biết secret. User không sửa được payload mà giữ chữ ký valid. Ngay cả khi user gửi field `role` trong body, Joi `stripUnknown: true` loại bỏ.

Bonus phòng: - Endpoint update profile cá nhân không cho update field `role` (Joi schema chỉ liệt kê `ho_ten`, `email`). - Endpoint update tài khoản (`PUT /api/accounts/:id`) chỉ cho `requireSuperAdmin`.

C.10 — Mass assignment

Ngắn: Tất cả Joi schema dùng `stripUnknown: true` để loại bỏ field thừa client gửi. Không bao giờ truyền `req.body` thẳng vào `prisma.create()`.

```
// Joi config
Joi.object({
  ho_ten: Joi.string().required(),
  email: Joi.string().email().optional(),
}).options({ stripUnknown: true });

// User gửi { ho_ten: 'A', role: 'ADMIN', is_super: true }
// → sau validate: { ho_ten: 'A' } — role và is_super bị strip
```

C.11 — Thông tin nhạy cảm trong response

- Không trả `password_hash` — Prisma `select: { id, username, role }` không kèm `password_hash`.
- Không trả refreshToken cho client (lưu trong DB và HttpOnly cookie cho FE thôi).
- Không trả CCCD đầy đủ cho USER — chỉ ADMIN/MANAGER xem được.

C.12 — DoS attack

Đa lớp: - `express-rate-limit`: 100 request / 15 phút / IP cho route công khai; 30 / 15 phút cho write endpoint. - `body-parser` limit JSON 10 MB → tránh JSON bomb. - File upload limit 10 MB. - Pagination forced: `MAX_LIMIT = 100` records/trang → không trả nhầm 100k records. - Slow loris: Nginx có timeout 30s.

Hạn chế: Layer 7 DDoS lớn cần WAF (Cloudflare) — không có vì project chạy LAN.

C.13 — Security headers

`helmet()` middleware đặt: - `X-Content-Type-Options: nosniff` - `X-Frame-Options: DENY` (chống clickjacking) - `Strict-Transport-Security` (HSTS) — bật khi có HTTPS - `X-XSS-Protection: 1; mode=block` - `Content-Security-Policy: default-src 'self'`

C.14 — CORS cấu hình thế nào?

```
// configs/cors.ts
export const corsOptions: CorsOptions = {
  origin: process.env.FRONTEND_URL || 'http://localhost:3000',
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
};
```

Whitelist 1 origin duy nhất → không cho domain khác gọi API.

C.15 — Audit log: ghi gì, không ghi gì?

Ghi: - `nguoi_thuc_hien_id`, `actor_role`, `action` (CREATE/UPDATE/DELETE/APPROVE/REJECT/IMPORT/...) - `resource` (vd: `personnel`, `proposals`) - `tai_nguyen_id` - `description` tiếng Việt - `payload` JSON: trạng thái before/after - `ip_address`, `user_agent`

Không ghi: - `password_hash` (kể cả khi UPDATE) - `refreshToken` - File content đính kèm (chỉ ghi tên file)

Resource `backup` ẩn với non-SUPER_ADMIN: trong `systemLogs.service.ts`:

```
if (userRole !== ROLES.SUPER_ADMIN) {
  whereClause.resource = { not: 'backup' };
}
```

C.16 — Reset password / quên mật khẩu

Hiện tại: chưa có flow self-service reset. SUPER_ADMIN/ADMIN dùng endpoint `POST /api/accounts/reset-password` (gửi `username` trong body) để reset thủ công.

Lý do: môi trường LAN nội bộ, người dùng có thể đến gặp ADMIN. Tránh phải chạy mail server trong LAN cô lập.

Hướng phát triển: thêm flow OTP qua SMS quân sự nội bộ.

D. Race condition và concurrency

D.1 — Hai user phê duyệt cùng đề xuất cùng lúc

Tình huống: Admin A và Admin B mở tab cùng đề xuất, bấm "Phê duyệt" gần như đồng thời.

Cơ chế chống: 1. **Transaction Prisma:** mở `prisma.$transaction(async tx => { ... })`. 2. **Re-fetch trong transaction với lock:** `tx.bangDeXuat.findUnique({ where: { id }, ... })` — Postgres mặc định READ COMMITTED, hai transaction đều đọc thấy `status = PENDING`. 3. **Kiểm tra status:** `if (proposal.status !== 'PENDING') throw new ConflictError('Đã được duyệt')`. 4. **Update có điều kiện:** `tx.bangDeXuat.updateMany({ where: { id, status: 'PENDING' }, data: { status: 'APPROVED' } })` — nếu trả về `count: 0` nghĩa là transaction kia đã update trước → throw conflict.

```
await prisma.$transaction(async tx => {
  const proposal = await tx.bangDeXuat.findUniqueOrThrow({ where: { id } });
  if (proposal.status !== PROPOSAL_STATUS.PENDING) {
    throw new ConflictError('Đề xuất đã được xử lý');
  }
  const updated = await tx.bangDeXuat.updateMany({
    where: { id, status: PROPOSAL_STATUS.PENDING },
    data: { status: PROPOSAL_STATUS.APPROVED, nguoi_duyet_id, ngay_duyet: new Date()
        },
  });
  if (updated.count === 0) {
    throw new ConflictError('Đề xuất vừa được người khác xử lý');
  }
  // ... gắn số quyết định, ghi nhật ký
});
```

Phản biện: “Sao không SELECT FOR UPDATE?” → “Có thể, nhưng `updateMany` có điều kiện đã đủ atomic ở mức row trong Postgres và không cần escape `$queryRaw`.”

D.2 — Tạo 2 tài khoản cùng username cùng lúc

Cơ chế: cột `TaiKhoan.username` có `@unique` → DB-level unique constraint. Hai INSERT cùng username sẽ có 1 thành công, 1 lỗi `P2002 (Unique constraint failed)`. Em catch và trả 409 Conflict.

```
try {
  return await prisma.taiKhoan.create({ data });
} catch (error) {
  if (error instanceof Prisma.PrismaClientKnownRequestError && error.code === 'P2002')
        {
    throw new ConflictError('Username đã tồn tại');
  }
  throw error;
}
```

Phản biện: “Sao không check trước rồi mới INSERT?” → “Check-then-insert vẫn race. Để DB unique làm trọng tài là chuẩn.”

D.3 — Import Excel: chèn dòng mới giữa preview và confirm

Tình huống: User upload Excel preview lúc 10:00. Lúc 10:01, có Admin khác tạo bản ghi `(quan_nhan_id, nam) = (X, 2025)` qua UI. Lúc 10:02, user bấm “Xác nhận import” — gặp dòng `(X, 2025)` trùng.

Cơ chế: - Bước “xác nhận” mở 1 transaction Prisma duy nhất. - Mỗi `tx.danhHieuHangNam.create({ data })` sẽ check constraint `@unique([quan_nhan_id, nam])`. - Nếu trùng → ném exception → toàn bộ transaction rollback. Không bản ghi nào vào DB. - Service trả về danh sách dòng lỗi cho user xem.

```
await prisma.$transaction(async tx => {
  for (const row of rows) {
    try {
      await tx.danhHieuHangNam.create({ data: row });
    } catch (e) {
      if (e.code === 'P2002') {
        throw new Error(`Dòng ${row._index}: trùng (quan_nhan_id=${row.quan_nhan_id},
        nam=${row.nam})`);
      }
      throw e;
    }
  }
});
```

D.4 — Đếm `so_luong` đơn vị khi 2 quân nhân chuyển đơn vị cùng lúc

Tình huống: Quân nhân A chuyển từ Đơn vị X → Y, quân nhân B chuyển từ Y → X cùng thời điểm. Nếu code dùng 2 lần `if`:

```
if (oldUnit) await unitRepo.decrement(oldUnit, 1);
if (newUnit) await unitRepo.increment(newUnit, 1);
```

Có thể đếm sai khi 2 transaction đan xen.

Cơ chế: - Mỗi update đơn vị trong **một transaction riêng** với atomic `increment/decrement` của Prisma. - Prisma sinh `UPDATE ... SET so_luong = so_luong + 1` — câu này atomic ở mức row trong Postgres.

```
await prisma.$transaction([
  prisma.coQuanDonVi.update({
    where: { id: oldUnit },
    data: { so_luong: { decrement: 1 } },
  }),
  prisma.coQuanDonVi.update({
    where: { id: newUnit },
    data: { so_luong: { increment: 1 } },
  }),
]);
```

Tuyệt đối tránh: đọc rồi tính rồi ghi:

```
// XẤU — race condition
const unit = await prisma.coQuanDonVi.findUnique({ where: { id: oldUnit } });
await prisma.coQuanDonVi.update({ where: { id: oldUnit }, data: { so_luong:
        unit.so_luong - 1 } });
```

Edge case quan trọng — chuyển nội bộ cùng CQDV: Nếu quân nhân chuyển từ DVTT-X sang DVTT-Y nhưng cùng CQDV cha → so_luong CQDV **không đổi**, chỉ DVTT đổi. Code đúng phải dùng `if/else`:

```
const isSameCqdv = oldDvtt?.co_quan_don_vi_id === newDvtt?.co_quan_don_vi_id;

if (!isSameCqdv) {
  // Đổi cả CQDV: decrement cũ, increment mới
  ops.push(prisma.coQuanDonVi.update({ where: { id: oldCqdv }, data: { so_luong: {
        decrement: 1 } } }));
  ops.push(prisma.coQuanDonVi.update({ where: { id: newCqdv }, data: { so_luong: {
        increment: 1 } } }));
}
// DVTT luôn đổi
ops.push(prisma.donViTrucThuoc.update({ where: { id: oldDvtt }, data: { so_luong: {
        decrement: 1 } } }));
ops.push(prisma.donViTrucThuoc.update({ where: { id: newDvtt }, data: { so_luong: {
        increment: 1 } } }));

await prisma.$transaction(ops);
```

Em đã có rule trong CLAUDE.md “Khi thay đổi đơn vị quân nhân, dùng if/else (chỉ increment/decrement 1 đơn vị), không dùng 2 if riêng biệt — tránh đếm dư”.

D.5 — Refresh token rotation race

Tình huống: Client gọi `/api/auth/refresh` 2 lần liên tiếp do retry → 2 refresh token mới được sinh, token nào mới hơn thắng.

Cơ chế trong project: - `accountRepository.update(id, { refreshToken: newToken })` là atomic UPDATE. - Nếu request 2 đến sau request 1: refreshToken ghi đè → request 1 còn dùng được token mới? Không, vì client chỉ giữ token cuối cùng nhận. - Server không reject token cũ cho đến khi expire (7 ngày) — đây là rủi ro.

Cải tiến đã ghi vào hướng phát triển: sliding window rotation với grace period 30s + blacklist.

D.6 — Backup chạy trùng

Cơ chế: Cron `backup.service.ts` dùng singleton flag trong process. Nếu cron tích hoạt khi job trước đang chạy → skip.

```
let isRunning = false;
async function runBackup() {
  if (isRunning) return;
  isRunning = true;
  try {
    await execPromise('pg_dump ...');
  } finally {
    isRunning = false;
  }
}
```

Hạn chế: flag chỉ trong 1 process. PM2 cluster mode sẽ có nhiều process → cần Redis lock. Hiện em chạy `pm2 ... -i 1` (single instance).

D.7 — Recalc trùng lặp gây sai dữ liệu

Cơ chế: `recalculateAnnualProfile(personnelId)` là **idempotent** — đọc dữ liệu nguồn → tính → upsert hồ sơ suy diễn `HoSoHangNam`. Chạy nhiều lần liên tiếp cho ra cùng kết quả.

```
async function recalculateAnnualProfile(personnelId) {
  const danhHieus = await prisma.danhHieuHangNam.findMany({ where: { quan_nhan_id:
        personnelId } });
  const context = computeChainContext(danhHieus);
  const flags = computeEligibilityFlags(context);
  await prisma.hoSoHangNam.upsert({
    where: { quan_nhan_id: personnelId },
    create: { quan_nhan_id: personnelId, ...flags },
    update: flags,
  });
}
```

`upsert` giải quyết race “create-or-update”.

D.8 — Approve và xoá đề xuất cùng lúc

Tình huống: ADMIN A bấm Phê duyệt; ADMIN B bấm Xoá cùng lúc.

Cơ chế: - Approve mở transaction → fetch → check status. Nếu lúc đó B đã DELETE: `findUniqueOrThrow` ném `P2025 (Record not found)` → throw `NotFoundError`. - Delete cũng dùng `deleteMany({ where: { id, status: 'PENDING' } })` — count = 0 nghĩa là proposal đã được duyệt → trả lỗi.

E. Logic chuỗi danh hiệu

E.1 — Giải thích rule chuỗi BKBQP / CSTĐTQ / BKTTCP cá nhân

Danh hiệu	Số năm chu kỳ	Cờ tiền điều kiện	Yêu cầu khác	Lifetime
BKBQP	2 năm CSTĐCS liên tục	—	NCKH mỗi năm trong chuỗi	Lặp lại
CSTĐTQ	3 năm CSTĐCS liên tục	≥ 1 BKBQP trong cửa sổ trượt 3 năm cuối	NCKH mỗi năm	Lặp lại
BKTTCP	7 năm CSTĐCS liên tục	đúng 3 BKBQP và đúng 2 CSTĐTQ trong 7 năm cuối	NCKH mỗi năm	Một lần duy nhất

E.2 — Cycle nghĩa là gì? Có gì khác lifetime?

- **Cycle (`isLifetime: false`):** danh hiệu lặp lại mỗi `cycleYears`. Eligibility = `streak >= cycleYears && streak % cycleYears === 0`. Vd: BKBQP nhận 2024 → đủ điều kiện đề nghị lần kế tiếp khi đạt CSTĐCS năm 2025 + 2026.
- **Lifetime (`isLifetime: true`):** chỉ nhận 1 lần. Sau khi nhận, hệ thống block với message "Đã có … chưa hỗ trợ cao hơn …". Áp dụng cho BKTTCP cá nhân.

E.3 — "Lỡ đợt" nghĩa là gì? Hệ thống xử lý ra sao?

Định nghĩa: Đến mốc đề nghị mà không có đề xuất → cycle tiếp tục đếm. Sau `cycleYears` năm sẽ lại đến mốc, vẫn được xét.

Ví dụ BKBQP cycle 2 năm: - 2023: CSTĐCS. 2024: CSTĐCS. → Đáng lẽ đề nghị BKBQP cuối 2024 nhưng "lỡ". - 2025: CSTĐCS. 2026: CSTĐCS. → Streak = 4, chia hết 2 → eligible cuối 2026.

Không cần "đứt chuỗi CSTĐCS" giữa chừng.

E.4 — Cửa sổ trượt 3 năm / 7 năm là gì?

Cửa sổ trượt 3 năm cho CSTĐTQ: đếm BKBQP trong 3 năm gần nhất tính từ `year - 1`. - Ví dụ xét CSTĐTQ năm 2026: cửa sổ từ 2023 đến 2025. - BKBQP nhận năm 2022 → ngoài cửa sổ → không đếm.

Cửa sổ 7 năm cho BKTTCP: đếm BKBQP và CSTĐTQ trong 7 năm gần nhất từ `year - 1`. - Personal BKTTCP yêu cầu **đúng** 3 BKBQP và **đúng** 2 CSTĐTQ vì lifetime. - Unit BKTTCP yêu cầu **≥ 3** BKBQP vì non-lifetime.

Vì sao tính từ `year - 1` chứ không phải `year`? Khi xét eligibility cho **năm hiện tại**, dữ liệu năm hiện tại có thể chưa đầy đủ (chưa có quyết định danh hiệu năm hiện tại). Cửa sổ tính từ `year - 1` nhìn về quá khứ — đảm bảo dữ liệu đã chốt.

Ví dụ cụ thể: Tháng 6/2026, xét CSTĐTQ cho năm 2026 → cửa sổ 3 năm là 2023, 2024, 2025. Nếu tính từ `year` thì cửa sổ là 2024-2026, mà 2026 chưa có dữ liệu danh hiệu hằng năm (thường ban hành cuối năm).

E.5 — `ChainContext` là gì?

Là object derive từ `DanhHieuHangNam` rows, không lưu DB. Chứa: - `chainStartYear`: năm bắt đầu chuỗi CSTĐCS hiện tại. - `lastBkbqpYear`, `lastCstdtqYear`, `lastBkttcpYear`: năm gần nhất nhận từng cờ. - `streakSinceLastBkbqp`, `streakSinceLastCstdtq`, `streakSinceLastBkttcp`: số năm CSTĐCS liên tục kể từ lần nhận gần nhất (0 nếu chưa từng nhận). - `missedBkbqp`, `missedCstdtq`: số chu kỳ đã lỡ.

Hàm `computeChainContext(danhHieus, currentYear)` đọc tất cả `DanhHieuHangNam` của 1 quân nhân và tính context. Phức tạp ~O(n) với n = số năm có dữ liệu.

E.6 — Tại sao tính context realtime mà không lưu DB?

Lý do: - Source of truth là `DanhHieuHangNam` — context chỉ là derive. - Nếu lưu DB → mỗi khi sửa danh hiệu phải invalidate cache, dễ lệch nhau. - Tính nhanh (~1 ms / quân nhân) → trade-off chấp nhận được.

Lưu vào `HoSoHangNam`: chỉ lưu **flags** đã quyết định (`du_dieu_kien_bkbqp/cstdtq/bkttcp`) cộng `goi_y` text — phục vụ filter danh sách. Khi sửa nguồn → gọi `recalculateAnnualProfile(id)` để cập nhật.

E.7 — Tại sao có `computeEligibilityFlags` (recalc) và `checkAwardEligibility` (API) — chỉ 1 không được sao?

Lý do tách: - `computeEligibilityFlags` chạy cho recalc → ghi flag vào DB cho **toàn bộ tier** (BKBQP, CSTĐTQ, BKTTCP). - `checkAwardEligibility` chạy cho 1 endpoint khi user định gửi 1 đề xuất cụ thể → chỉ check 1 tier + trả message lỗi cụ thể.

Đảm bảo nhất quán: Cả hai gọi chung `chainEligibility.checkChainEligibility` cho rule core. Khác nhau ở phần "lifetime block" mà chỉ recalc cần xử lý (vì recalc set `goi_y` cho UI).

E.8 — Test case cho rule chuỗi

Có 197 ca kiểm thử riêng cho phần này, phân bổ: - `eligibility-bkbqp-personal.test.ts` — 95 ca: vừa đủ chu kỳ, lỡ chu kỳ, lặp chu kỳ, NCKH thiếu, NCKH có nhưng CSTĐCS đứt. - `eligibility-cstdtq-personal.test.ts` — 80 ca: cửa sổ trượt 3 năm có/không có BKBQP, BKBQP rơi khỏi cửa sổ. - `eligibility-bkttcp-personal.test.ts` — 60 ca: lifetime block, đếm `=== 3` BKBQP và `=== 2` CSTĐTQ strict. - `eligibility-bkbqp-unit.test.ts` — 50 ca: ĐVQT 2 năm. - `eligibility-bkttcp-

`unit.test.ts` — 45 ca: cửa sổ trượt 7 năm, non-lifetime. - `chainContext.test.ts` — 30 ca: derive context. - `chainCycleScenarios.test.ts` — 80 ca: scenarios tổng hợp.

F. Cú pháp Prisma đối chiếu SQL

F.0 — Setup Prisma client

```
// models/index.ts
import { PrismaClient } from '../generated/prisma';
export const prisma = new PrismaClient({
  log: process.env.NODE_ENV === 'development' ? ['query', 'error'] : ['error'],
});
```

Bật `log: ['query']` để xem SQL Prisma sinh ra trong console khi debug.

F.1 — Tìm 30 log gần nhất theo loại action ‘A’ trong 30 ngày qua

Prisma:

```
const days30Ago = new Date(Date.now() - 30 * 24 * 60 * 60 * 1000);

const logs = await prisma.systemLog.findMany({
  where: {
    action: 'CREATE',
    createdAt: { gte: days30Ago },
  },
  orderBy: { createdAt: 'desc' },
  take: 30,
  select: {
    id: true,
    actor_role: true,
    description: true,
    createdAt: true,
    NguoiThucHien: { select: { username: true } },
  },
});
```

SQL tương đương:

```sql
SELECT
  l.id,
  l.actor_role,
  l.description,
  l."createdAt",
  t.username AS nguoi_thuc_hien
FROM "SystemLog" l
LEFT JOIN "TaiKhoan" t ON t.id = l.nguoi_thuc_hien_id
WHERE l.action = 'CREATE'
  AND l."createdAt" >= NOW() - INTERVAL '30 days'
ORDER BY l."createdAt" DESC
LIMIT 30;
```

Index hỗ trợ: `@@index([action, createdAt])` đã có trong schema → query dùng index B-tree, ~O(log n).

F.2 — Tìm 30 đề xuất loại A hoặc B mới nhất

Prisma:

```prisma
const proposals = await prisma.bangDeXuat.findMany({
  where: {
    loai_de_xuat: { in: ['CA_NHAN_HANG_NAM', 'NIEN_HAN'] },
  },
  orderBy: { createdAt: 'desc' },
  take: 30,
  include: {
    NguoiDeXuat: { select: { id: true, username: true } },
    CoQuanDonVi: { select: { ten_don_vi: true } },
  },
});
```

SQL:

```sql
SELECT bdx.*, t.username, cqdv.ten_don_vi
FROM "BangDeXuat" bdx
LEFT JOIN "TaiKhoan" t ON t.id = bdx.nguoi_de_xuat_id
LEFT JOIN "CoQuanDonVi" cqdv ON cqdv.id = bdx.co_quan_don_vi_id
WHERE bdx.loai_de_xuat IN ('CA_NHAN_HANG_NAM', 'NIEN_HAN')
ORDER BY bdx."createdAt" DESC
LIMIT 30;
```

F.3 — Tìm log theo nhiều bộ lọc kết hợp (date range + action + role + resource + keyword)

Prisma:

```
const where: Prisma.SystemLogWhereInput = {
  AND: [
    { createdAt: { gte: from, lte: to } },
    actions.length > 0 ? { action: { in: actions } } : {},
    role ? { actor_role: role } : {},
    resource ? { resource } : {},
    keyword ? { description: { contains: keyword, mode: 'insensitive' } } : {},
  ],
};

const [data, total] = await Promise.all([
  prisma.systemLog.findMany({
    where,
    orderBy: { createdAt: 'desc' },
    skip: (page - 1) * limit,
    take: limit,
  }),
  prisma.systemLog.count({ where }),
]);
```

SQL:

```
-- Đếm tổng
SELECT COUNT(*) FROM "SystemLog"
WHERE "createdAt" BETWEEN $1 AND $2
  AND action = ANY($3)
  AND actor_role = $4
  AND resource = $5
  AND description ILIKE '%' || $6 || '%';

-- Lấy data
SELECT * FROM "SystemLog"
WHERE "createdAt" BETWEEN $1 AND $2
  AND action = ANY($3)
  AND actor_role = $4
  AND resource = $5
  AND description ILIKE '%' || $6 || '%'
ORDER BY "createdAt" DESC
LIMIT $7 OFFSET $8;
```

Lưu ý: Em dùng `Promise.all` chạy song song count + data → giảm latency. Nếu chạy tuần tự sẽ tốn gấp đôi thời gian.

F.4 — Pagination với cursor (cho bảng > 100k rows)

Prisma offset (đơn giản, chậm khi page sâu):

```
const data = await prisma.quanNhan.findMany({
  skip: (page - 1) * limit,
  take: limit,
  orderBy: { id: 'asc' },
});
```

Prisma cursor (nhanh hơn ở page sâu):

```
const data = await prisma.quanNhan.findMany({
  cursor: lastId ? { id: lastId } : undefined,
  skip: lastId ? 1 : 0,
  take: limit,
  orderBy: { id: 'asc' },
});
const nextCursor = data[data.length - 1]?.id;
```

SQL offset (giáo trình):

```
SELECT * FROM "QuanNhan" ORDER BY id LIMIT 50 OFFSET 1000;  -- chậm khi offset lớn
```

SQL cursor (production):

```
SELECT * FROM "QuanNhan" WHERE id > 'last_id' ORDER BY id LIMIT 50;
```

Ngữ cảnh project: Hiện em dùng offset vì dataset nhỏ (< 5k quân nhân). Khi vượt 100k records sẽ chuyển cursor.

F.5 — Đếm số quân nhân theo đơn vị (group by)

Prisma:

```
const counts = await prisma.quanNhan.groupBy({
  by: ['co_quan_don_vi_id'],
  _count: { id: true },
  orderBy: { _count: { id: 'desc' } },
});
```

SQL:

```
SELECT co_quan_don_vi_id, COUNT(id) AS _count_id
FROM "QuanNhan"
GROUP BY co_quan_don_vi_id
ORDER BY _count_id DESC;
```

F.6 — Thống kê khen thưởng theo năm và loại danh hiệu

Prisma raw query (vì cần CASE WHEN):

```
const stats = await prisma.$queryRaw<Array<{ nam: number; danh_hieu: string; so_luong:
        bigint }>>`
  SELECT nam, danh_hieu, COUNT(*) AS so_luong
  FROM "DanhHieuHangNam"
  WHERE nam BETWEEN ${fromYear} AND ${toYear}
  GROUP BY nam, danh_hieu
  ORDER BY nam DESC, danh_hieu;
`;
```

Convert bigint → number:

```
const result = stats.map(s => ({ ...s, so_luong: Number(s.so_luong) }));
```

F.7 — Find quân nhân thuộc cây đơn vị MANAGER quản lý

Prisma:

```
const personnelInUnit = await prisma.quanNhan.findMany({
  where: {
    OR: [
      { co_quan_don_vi_id: managerCqdvId },
      { don_vi_truc_thuoc_id: { in: dvttIdsUnderCqdv } },
    ],
  },
  include: { ChucVu: true, CoQuanDonVi: true, DonViTrucThuoc: true },
});
```

SQL:

```
SELECT qn.*, cv.ten_chuc_vu, cqdv.ten_don_vi AS cqdv_ten, dvtt.ten_don_vi AS dvtt_ten
FROM "QuanNhan" qn
LEFT JOIN "ChucVu" cv ON cv.id = qn.chuc_vu_id
LEFT JOIN "CoQuanDonVi" cqdv ON cqdv.id = qn.co_quan_don_vi_id
LEFT JOIN "DonViTrucThuoc" dvtt ON dvtt.id = qn.don_vi_truc_thuoc_id
WHERE qn.co_quan_don_vi_id = $1
   OR qn.don_vi_truc_thuoc_id = ANY($2);
```

F.8 — Eager loading nested 3 cấp

Prisma:

```
const quanNhan = await prisma.quanNhan.findUnique({
  where: { id },
  include: {
    DonViTrucThuoc: {
      include: {
        CoQuanDonVi: true,
      },
    },
    DanhHieuHangNam: {
      orderBy: { nam: 'desc' },
      take: 10,
    },
    LichSuChucVu: {
      include: { ChucVu: true },
      orderBy: { ngay_bat_dau: 'desc' },
    },
  },
});
```

SQL: Prisma sinh nhiều query JOIN. Có thể debug bằng `log: ['query']` trong client config.

F.9 — Insert batch nhiều bản ghi

Prisma `createMany` (nhanh nhất):

```
await prisma.danhHieuHangNam.createMany({
  data: rows.map(r => ({ quan_nhan_id: r.id, nam: r.nam, danh_hieu: r.code })),
  skipDuplicates: true,  // bỏ qua row vi phạm @unique
});
```

SQL:

```
INSERT INTO "DanhHieuHangNam" (id, quan_nhan_id, nam, danh_hieu)
VALUES (gen_random_uuid(), $1, $2, $3),
       (gen_random_uuid(), $4, $5, $6),
       ...
ON CONFLICT (quan_nhan_id, nam) DO NOTHING;
```

Hạn chế `createMany`: không trả về danh sách đã tạo (Postgres). Em phải query lại nếu cần ID.

F.10 — Update có điều kiện (atomic)

Prisma `updateMany`:

```
const result = await prisma.bangDeXuat.updateMany({
  where: { id, status: 'PENDING' },
  data: { status: 'APPROVED', nguoi_duyet_id, ngay_duyet: new Date() },
});
if (result.count === 0) throw new ConflictError('Đã được người khác xử lý');
```

SQL:

```sql
UPDATE "BangDeXuat"
SET status = 'APPROVED', nguoi_duyet_id = $1, ngay_duyet = NOW()
WHERE id = $2 AND status = 'PENDING'
RETURNING id;
-- Nếu không có row trả về → đã bị update bởi transaction khác
```

F.11 — Upsert (insert or update)

Prisma:

```prisma
await prisma.hoSoHangNam.upsert({
  where: { quan_nhan_id: personnelId },
  create: { quan_nhan_id: personnelId, ...flags },
  update: flags,
});
```

SQL:

```sql
INSERT INTO "HoSoHangNam" (quan_nhan_id, du_dieu_kien_bkbqp, ...)
VALUES ($1, $2, ...)
ON CONFLICT (quan_nhan_id) DO UPDATE
SET du_dieu_kien_bkbqp = EXCLUDED.du_dieu_kien_bkbqp;
```

F.12 — Transaction nhiều thao tác

Prisma sequential transaction (array):

```prisma
await prisma.$transaction([
  prisma.bangDeXuat.update({ where: { id }, data: { status: 'APPROVED' } }),
  prisma.danhHieuHangNam.createMany({ data: rows }),
  prisma.systemLog.create({ data: logEntry }),
]);
```

Prisma interactive transaction (linh hoạt hơn):

```prisma
await prisma.$transaction(async tx => {
  const proposal = await tx.bangDeXuat.findUnique({ where: { id } });
  if (proposal.status !== 'PENDING') throw new ConflictError();
  await tx.bangDeXuat.update({ where: { id }, data: { status: 'APPROVED' } });
  await tx.danhHieuHangNam.createMany({ data: rows });
  await tx.systemLog.create({ data: logEntry });
});
```

SQL:

```sql
BEGIN;
UPDATE "BangDeXuat" SET status='APPROVED' WHERE id=$1;
INSERT INTO "DanhHieuHangNam" (...) VALUES (...);
INSERT INTO "SystemLog" (...) VALUES (...);
COMMIT;
-- Nếu một câu lỗi: ROLLBACK;
```

F.13 — Find với JSON field (PostgreSQL JSONB)

Prisma — query JSONB path:

```typescript
const logs = await prisma.systemLog.findMany({
  where: {
    payload: {
      path: ['before', 'status'],
      equals: 'PENDING',
    },
  },
});
```

SQL:

```sql
SELECT * FROM "SystemLog"
WHERE payload #>> '{before,status}' = 'PENDING';
```

F.14 — Đếm theo điều kiện (counted aggregation)

Prisma:

```typescript
const count = await prisma.danhHieuHangNam.count({
  where: { quan_nhan_id, danh_hieu: 'CSTDCS', nam: { gte: 2020, lte: 2024 } },
});
```

SQL:

```sql
SELECT COUNT(*) FROM "DanhHieuHangNam"
WHERE quan_nhan_id = $1 AND danh_hieu = 'CSTDCS' AND nam BETWEEN 2020 AND 2024;
```

F.15 — Aggregate (sum, avg, min, max)

Prisma:

```
const stat = await prisma.lichSuChucVu.aggregate({
  where: { quan_nhan_id },
  _sum: { so_thang: true },
  _avg: { he_so_chuc_vu: true },
  _min: { ngay_bat_dau: true },
  _max: { ngay_bat_dau: true },
});
```

SQL:

```
SELECT
  SUM(so_thang) AS sum_so_thang,
  AVG(he_so_chuc_vu) AS avg_he_so,
  MIN(ngay_bat_dau) AS min_start,
  MAX(ngay_bat_dau) AS max_start
FROM "LichSuChucVu"
WHERE quan_nhan_id = $1;
```

F.16 — Distinct values

Prisma:

```
const years = await prisma.danhHieuHangNam.findMany({
  distinct: ['nam'],
  select: { nam: true },
  orderBy: { nam: 'desc' },
});
```

SQL:

```
SELECT DISTINCT nam FROM "DanhHieuHangNam" ORDER BY nam DESC;
```

F.17 — Find với LIKE (search keyword)

Prisma:

```
const result = await prisma.quanNhan.findMany({
  where: {
    OR: [
      { ho_ten: { contains: keyword, mode: 'insensitive' } },
      { cccd: { contains: keyword } },
      { so_dien_thoai: { contains: keyword } },
    ],
  },
});
```

SQL:

```
SELECT * FROM "QuanNhan"
WHERE ho_ten ILIKE '%' || $1 || '%'
   OR cccd LIKE '%' || $1 || '%'
   OR so_dien_thoai LIKE '%' || $1 || '%';
```

F.18 — Soft delete pattern (giả sử)

Schema chưa dùng nhưng nếu cần:

```
// Filter chỉ lấy không bị xoá
const active = await prisma.quanNhan.findMany({
  where: { deletedAt: null },
});

// "Xoá" = set deletedAt
await prisma.quanNhan.update({
  where: { id },
  data: { deletedAt: new Date() },
});
```

F.19 — Cascade delete

Schema:

```
model QuanNhan {
  id String @id
  TaiKhoan TaiKhoan?
  DanhHieuHangNam DanhHieuHangNam[]
}
model TaiKhoan {
  quan_nhan_id String? @unique
  QuanNhan QuanNhan? @relation(fields: [quan_nhan_id], references: [id], onDelete:
Cascade)
}
```

Khi xoá QuanNhan: Postgres tự xoá TaiKhoan, DanhHieuHangNam liên quan.

Tránh accidentally cascade: Dùng `onDelete: Restrict` cho FK quan trọng (vd: `FileQuyetDinh` không cho xoá khi còn bản ghi tham chiếu).

F.20 — Transaction với isolation level

Prisma (mặc định READ COMMITTED):

```
await prisma.$transaction(
  async tx => { ... },
  { isolationLevel: Prisma.TransactionIsolationLevel.Serializable }
);
```

Khi nào cần `Serializable`? Khi rule yêu cầu "không có 2 transaction đồng thời ghi đè nhau" — vd: kiểm phong tỏa số quyết định trùng. Trade-off: chậm hơn 2-3 lần do Postgres phải retry.

F.21 — Window function (rank, lag)

Prisma không hỗ trợ window function tự nhiên → dùng `$queryRaw`:

```
const ranked = await prisma.$queryRaw<Array<{ id: string; rank: bigint }>>`
  SELECT id, ho_ten,
    RANK() OVER (PARTITION BY co_quan_don_vi_id ORDER BY (
      SELECT COUNT(*) FROM "DanhHieuHangNam" dh WHERE dh.quan_nhan_id = qn.id AND
        dh.danh_hieu = 'CSTDCS'
    ) DESC) AS rank
  FROM "QuanNhan" qn;
`;
```

SQL: giống raw query trên, parameterize.

F.22 — CTE (Common Table Expression)

```
await prisma.$queryRaw`
  WITH personnel_with_streak AS (
    SELECT quan_nhan_id, COUNT(*) AS streak
    FROM "DanhHieuHangNam"
    WHERE danh_hieu = 'CSTDCS' AND nam >= ${currentYear - 2}
    GROUP BY quan_nhan_id
    HAVING COUNT(*) >= 2
  )
  SELECT qn.*, p.streak FROM "QuanNhan" qn
  JOIN personnel_with_streak p ON p.quan_nhan_id = qn.id;
`;
```

F.23 — Filter theo relation field

Prisma — quân nhân có ít nhất 1 BKBQP từ năm 2020:

```
const result = await prisma.quanNhan.findMany({
  where: {
    DanhHieuHangNam: {
      some: { nhan_bkbqp: true, nam: { gte: 2020 } },
    },
  },
});
```

SQL:

```
SELECT DISTINCT qn.* FROM "QuanNhan" qn
JOIN "DanhHieuHangNam" dh ON dh.quan_nhan_id = qn.id
WHERE dh.nhan_bkbqp = TRUE AND dh.nam >= 2020;
```

F.24 — Filter ngược (none)

Prisma — quân nhân chưa từng nhận BKBQP:

```
const result = await prisma.quanNhan.findMany({
  where: {
    DanhHieuHangNam: { none: { nhan_bkbqp: true } },
  },
});
```

SQL:

```
SELECT qn.* FROM "QuanNhan" qn
WHERE NOT EXISTS (
  SELECT 1 FROM "DanhHieuHangNam" dh
  WHERE dh.quan_nhan_id = qn.id AND dh.nhan_bkbqp = TRUE
);
```

F.25 — Filter với every (all relations match)

Prisma — quân nhân mọi năm đều có CSTĐCS:

```
const result = await prisma.quanNhan.findMany({
  where: {
    DanhHieuHangNam: { every: { danh_hieu: 'CSTDCS' } },
  },
});
```

F.26 — Update many với increment

Prisma:

```
await prisma.coQuanDonVi.update({
  where: { id },
  data: { so_luong: { increment: 1 } },
});
```

SQL:

```
UPDATE "CoQuanDonVi" SET so_luong = so_luong + 1 WHERE id = $1;
```

F.27 — Connect / disconnect relation

Prisma — gắn quân nhân vào tài khoản:

```
await prisma.taiKhoan.update({
  where: { id: accountId },
  data: { QuanNhan: { connect: { id: quanNhanId } } },
});
```

SQL:

```
UPDATE "TaiKhoan" SET quan_nhan_id = $1 WHERE id = $2;
```

F.28 — Disconnect

Prisma:

```
await prisma.taiKhoan.update({
  where: { id },
  data: { QuanNhan: { disconnect: true } },
});
```

SQL:

```
UPDATE "TaiKhoan" SET quan_nhan_id = NULL WHERE id = $1;
```

F.29 — Nested write (tạo cha + con cùng lúc)

Prisma:

```
await prisma.quanNhan.create({
  data: {
    ho_ten: 'Nguyễn Văn A',
    chuc_vu_id: 'cv001',
    DanhHieuHangNam: {
      create: [
        { nam: 2023, danh_hieu: 'CSTDCS' },
        { nam: 2024, danh_hieu: 'CSTDCS' },
      ],
    },
  },
});
```

SQL: Prisma sinh `INSERT QuanNhan ... RETURNING id` rồi `INSERT DanhHieuHangNam (..., quan_nhan_id) VALUES ...`. Nằm trong cùng transaction.

F.30 — Find unique theo composite key

Prisma:

```
const dh = await prisma.danhHieuHangNam.findUnique({
  where: { quan_nhan_id_nam: { quan_nhan_id, nam } },  // tên key sinh từ
        @unique([quan_nhan_id, nam])
});
```

SQL:

```
SELECT * FROM "DanhHieuHangNam" WHERE quan_nhan_id = $1 AND nam = $2;
```

F.31 — Truy vấn với raw + Prisma typed

```
const result = await prisma.$queryRaw<Array<{ id: string; total: bigint }>>`
  SELECT quan_nhan_id AS id, COUNT(*) AS total
  FROM "DanhHieuHangNam"
  WHERE danh_hieu = 'CSTDCS' AND nam >= ${fromYear}
  GROUP BY quan_nhan_id
  HAVING COUNT(*) >= ${minCount};
`;
```

`<>` là explicit type cho TypeScript autocomplete.

F.32 — Find với date range hỗn hợp

Prisma:

```
const logs = await prisma.systemLog.findMany({
  where: {
    createdAt: {
      gte: startOfDay(from),
      lte: endOfDay(to),
    },
  },
});
```

SQL:

```
SELECT * FROM "SystemLog" WHERE "createdAt" >= $1 AND "createdAt" <= $2;
```

F.33 — Truy vấn join nhiều bảng cộng aggregation

```
// "Top 10 đơn vị có nhiều đề xuất chờ duyệt nhất"
const top = await prisma.$queryRaw<Array<{ ten_don_vi: string; pending: bigint }>>`
  SELECT cqdv.ten_don_vi, COUNT(bdx.id) AS pending
  FROM "CoQuanDonVi" cqdv
  LEFT JOIN "BangDeXuat" bdx ON bdx.co_quan_don_vi_id = cqdv.id AND bdx.status =
        'PENDING'
  GROUP BY cqdv.id, cqdv.ten_don_vi
  HAVING COUNT(bdx.id) > 0
  ORDER BY pending DESC
  LIMIT 10;
`;
```

F.34 — N+1 query — anti-pattern

XẤU:

```
const quanNhans = await prisma.quanNhan.findMany();
for (const qn of quanNhans) {
  qn.danhHieus = await prisma.danhHieuHangNam.findMany({ where: { quan_nhan_id: qn.id
        } });
}
// → 1 + N query
```

TỐT (eager loading):

```
const quanNhans = await prisma.quanNhan.findMany({
  include: { DanhHieuHangNam: true },
});
// → 2 query, JOIN trong query 2
```

TỐT HƠN (batch with Map):

```
const quanNhans = await prisma.quanNhan.findMany();
const ids = quanNhans.map(q => q.id);
const allDanhHieus = await prisma.danhHieuHangNam.findMany({
  where: { quan_nhan_id: { in: ids } },
});
const danhHieuMap = new Map<string, DanhHieuHangNam[]>();
for (const dh of allDanhHieus) {
  const arr = danhHieuMap.get(dh.quan_nhan_id) || [];
  arr.push(dh);
  danhHieuMap.set(dh.quan_nhan_id, arr);
}
quanNhans.forEach(q => (q.danhHieus = danhHieuMap.get(q.id) || []));
// → 2 query, không có JOIN nặng
```

F.35 — Promise.all cho query song song

```
const [quanNhanCount, donViCount, deXuatPending] = await Promise.all([
  prisma.quanNhan.count(),
  prisma.coQuanDonVi.count(),
  prisma.bangDeXuat.count({ where: { status: 'PENDING' } }),
]);
```

3 query chạy song song → tổng thời gian ≈ max(t1, t2, t3) thay vì t1+t2+t3.

G. Hiệu năng và mở rộng

G.1 — Hệ thống chịu được bao nhiêu user concurrent?

Trả lời: Em chưa load test cụ thể. Ước tính theo benchmark Express + Prisma + Postgres trên server 4 GB RAM: - ~500 request/giây cho endpoint đọc đơn giản (find by id). - ~100 request/giây cho endpoint phức tạp (recalc). - Số user concurrent ước tính 200-300.

Cách scale: PM2 cluster mode (`pm2 ... -i max`) → 1 process / CPU core. 4 cores = 4 process. Cần Redis cho session store nếu cluster.

Hiện tại: chạy single instance vì user thực tế ~50 người LAN nội bộ → không cần cluster.

G.2 — Endpoint nào chậm nhất? Tối ưu ra sao?

Hiện tại chậm nhất: `POST /api/profiles/recalculate-all` — chạy `recalculateAnnualProfile` cho 1247 quân nhân tốn 18 giây.

Đã tối ưu: - Dùng `Promise.all` chạy song song theo batch 50 quân nhân/lần. - Mỗi `recalculateAnnualProfile` chỉ ~14 ms. - Có thể cache kết quả nếu dataset chưa đổi.

Có thể cải thiện thêm: - Worker thread riêng (`worker_threads` Node.js). - Hàng đợi (BullMQ + Redis) cho job dài.

G.3 — Index database

Index có sẵn theo schema: - `@@unique([quan_nhan_id, nam])` cho `DanhHieuHangNam` → composite unique index. - `@@index([co_quan_don_vi_id])` cho `QuanNhan`. - `@@index([action, createdAt])`, `@@index([resource, createdAt])`, `@@index([actor_role, createdAt])` cho `SystemLog` → tối ưu cho 3 dimension lọc phổ biến. - `@@index([so_quyet_dinh])` cho mọi bảng output → tối ưu lookup theo số quyết định.

Cần thêm khi scale: index trên `BangDeXuat.createdAt` + `loai_de_xuat` cho filter list.

G.4 — Database connection pooling

Prisma mặc định pool size = `num_physical_cpus * 2 + 1`. Với server 4 cores → pool = 9.

Có thể chỉnh trong `DATABASE_URL`:

```
postgresql://user:pass@localhost:5432/db?connection_limit=20&pool_timeout=10
```

G.5 — Bundle size frontend

Next.js tự code-split theo route. Bundle initial ~ 250 KB gzipped.

Tối ưu đã làm: - Lazy import Ant Design component khi cần (`import { Table } from 'antd'` thay vì `import * as antd`). - Tailwind purge content `'./src/**/*.{js,ts,jsx,tsx}'`. - Image optimization của Next.js (`next/image`).

G.6 — Caching

Chưa có cache layer riêng. Hiện tại mọi query trực tiếp DB.

Hướng phát triển: Redis cache cho: - Permission lookup (~5 ms saving / request). - Dashboard statistics (refresh mỗi 5 phút). - Đơn vị tree (ít thay đổi, có thể cache 1 giờ).

H. Kiểm thử

H.1 — Tỉ lệ unit test bao nhiêu là đủ?

Trả lời: Mục tiêu của em là cover 100 % rule logic phức tạp (chuỗi danh hiệu, eligibility), > 80 % service layer, ≥ 70 % overall.

Hiện tại: - 870 test cases / 74 suites pass 100 %. - Coverage ≥ 85 % cho `services/profile`, `services/eligibility`, `services/proposal`. - Một số helper pure function 100 %. - Controller layer thấp hơn (~60 %) — em ưu tiên test logic hơn integration.

H.2 — Mock Prisma thế nào?

```
// tests/setup.ts
import { mockDeep, mockReset } from 'jest-mock-extended';
import { PrismaClient } from '../src/generated/prisma';

export const prismaMock = mockDeep<PrismaClient>();
jest.mock('../src/models', () => ({ prisma: prismaMock }));

beforeEach(() => mockReset(prismaMock));

// Trong test
prismaMock.quanNhan.findMany.mockResolvedValue([{ id: '1', ho_ten: 'A' }] as any);
```

H.3 — Khác giữa unit test và integration test

Loại	Phạm vi	Tốc độ	Mock	Đại diện
Unit	1 hàm pure	Rất nhanh (ms)	Không cần	`chainEligibility.test.ts`
Service unit	1 service + mock Prisma	Nhanh	Có	`proposalService.test.ts`
Integration	Nhiều layer + DB thật	Chậm (giây)	Không	`tests/scenarios/...`

Project em chủ yếu unit + service unit, có ~10 integration test trong `tests/scenarios/` chạy trên DB test riêng.

H.4 — TDD không? Code-first hay test-first?

Trả lời: Em không TDD strict. Quy trình thường là: 1. Viết code logic (~1 giờ). 2. Viết test cases ngay sau đó (~30 phút) — chủ yếu cho rule chuỗi. 3. Bug fix → bắt buộc viết test reproduce trước, sau đó fix.

Trường hợp TDD: rule eligibility chuỗi (197 ca test) — em viết test case dạng table-driven trước, sau đó implement.

I. Triển khai và vận hành

I.1 — Quy trình deploy production

1. Pull code mới về server.
2. `cd BE-QLKT && npm ci && npm run build && npx prisma migrate deploy`.
3. `cd FE-QLKT && npm ci && npm run build`.
4. `pm2 reload ecosystem.config.js` — zero-downtime reload.
5. Kiểm tra health check: `curl http://localhost:4000/health`.

I.2 — Rollback nếu deploy lỗi

1. Backup DB ngay trước migrate (cron tự chạy 2:00 sáng + manual trigger qua DevZone).
2. `pm2 reload --revert` để quay lại version cũ.
3. Nếu migrate đã chạy → restore DB từ backup `pg_dump` (~14 giây cho dataset hiện tại).

I.3 — Monitor production

Hiện tại: - PM2 monitor (`pm2 logs`, `pm2 monit`). - Error log ghi vào `BE-QLKT/logs/error.log` qua winston. - System log trong DB cho audit.

Hướng phát triển: Sentry cho error tracking, Grafana + Prometheus cho metrics.

I.4 — Backup chiến lược

- **Cron tự động:** 02:00 sáng hằng ngày. Lưu vào `backups/<timestamp>.sql` (SQL text format từ `pg_dump --format=plain`).
- **Retention:** 30 ngày. File cũ tự xóa qua `cleanup` cron.
- **Manual trigger:** SUPER_ADMIN qua DevZone.
- **Test restore:** thử khôi phục mỗi tháng 1 lần, mất ~14 giây cho dataset hiện tại.

I.5 — Khi server crash

1. PM2 auto-restart 3 lần liên tục (config `max_restarts: 3, restart_delay: 5000`).
2. Nếu vẫn crash: log Slack/Telegram (chưa setup).
3. Health check Nginx report 502 → tạm hiển thị trang "Hệ thống đang bảo trì".

J. Câu hỏi khoai và edge case

J.1 — Hệ thống chống trùng dữ liệu ở những đâu?

Các điểm check trùng trong project (đã verify trong code):

Dữ liệu	Cột unique	File chặn ở tầng service	Message
CCCD quân nhân	`QuanNhan.cccd @unique`	`personnel.service.ts:202, 417`	"CCCD đã tồn tại trong hệ thống"
Username tài khoản	`TaiKhoan.username @unique`	`account.service.ts:232`, `personnel.service.ts:232`	"Tên đăng nhập đã tồn tại" / "Username (CCCD) đã tồn tại trong hệ thống tài khoản"
Số quyết định	`FileQuyetDinh.so_quyet_dinh @unique`	`decision.service.ts:334, 379`	"Số quyết định đã tồn tại" (409)
Mã đơn vị	`CoQuanDonVi.ma_don_vi @unique`, `DonViTrucThuoc.ma_don_vi @unique`	`unit.service.ts:89, 130, 154`	"Mã đơn vị đã tồn tại" (409)
Tên chức vụ trong cùng đơn vị	`@@unique([co_quan_don_vi_id, ten_chuc_vu])`	`position.service.ts:74`	"Tên chức vụ đã tồn tại trong đơn vị này" (409)
Thành tích NCKH	composite unique	`nckhStrategy.ts:140`, `proposal/approve/validation.ts:189`	"Thành tích '…' năm X đã tồn tại"
Danh hiệu hằng năm	`@@unique([quan_nhan_id, nam])`	DB-level + import preview	bắt qua P2002

Lưu ý quan trọng — Username = CCCD: Khi tạo tài khoản cho quân nhân, username chính là CCCD (`personnel.service.ts:232` ghi rõ "Username (CCCD)"). Vì vậy check trùng username thực chất là check CCCD ở tầng tài khoản — đảm bảo 1 CCCD chỉ có tối đa 1 tài khoản đăng nhập.

Cơ chế chống trùng 2 lớp (defense in depth): 1. **Service layer:** Trước khi `prisma.create`, em `findFirst` check trùng → trả message thân thiện tiếng Việt. 2. **DB layer:** Cột `@unique` chặn cuối cùng. Nếu race condition lọt qua check service, INSERT trùng → Postgres trả `P2002` → middleware `errorHandler.ts:71` trả "Dữ liệu đã tồn tại (trùng lặp)".

Trường hợp NULL: Cột `cccd` nullable. Hai quân nhân cùng `cccd = NULL` không bị conflict (Postgres coi NULL ≠ NULL trong unique constraint). Em chấp nhận điều này — quân nhân chưa có CCCD vẫn tạo được hồ sơ.

KHÔNG có check trùng `ho_ten + ngay_sinh`: Việt Nam có nhiều người trùng tên + ngày sinh (vd: "Nguyễn Văn An" 01/01/2000) → khoá phân biệt thật là CCCD. Em chỉ unique theo CCCD, cho phép trùng tên + ngày sinh.

J.2 — Quân nhân chuyển đơn vị giữa năm có ảnh hưởng eligibility không?

Trả lời: Không. Eligibility chuỗi danh hiệu tính theo cờ trong `DanhHieuHangNam` (`nhan_bkbqp/cstdtq/bkttcp` + năm) — không phụ thuộc đơn vị tại thời điểm xét.

Ảnh hưởng cách hiển thị: Sau khi chuyển đơn vị, `DanhHieuHangNam` của năm cũ có thể có `cap_bac/chuc_vu` lưu snapshot. Nhưng chuỗi vẫn tính đúng.

J.3 — Năm sau hệ thống đổi rule eligibility (vd: BKBQP cycle 3 năm thay vì 2)?

Cơ chế: Đổi `cycleYears` trong `PERSONAL_CHAIN_AWARDS[BKBQP]` của `chainAwards.constants.ts`. Toàn bộ hệ thống tự áp rule mới.

Cần làm thêm: 1. Chạy `recalculateAnnualProfile` cho tất cả quân nhân để cập nhật flag `du_dieu_kien_*`. 2. Update test cases cho rule mới. 3. Migration data nếu cần (vd: vô hiệu hoá đề xuất pending bị mất hiệu lực).

J.4 — Backup file bị xoá nhầm — lấy lại được không?

Cơ chế bảo vệ: - Backup lưu trên cùng server BE → nếu disk hỏng thì mất. - **Hướng phát triển:** rsync sang NAS nội bộ Học viện, encrypted.

Hiện tại: chấp nhận rủi ro do hạ tầng LAN cô lập, không có cloud.

J.5 — Nếu Prisma migration fail giữa chừng

Cơ chế: Mỗi migration là 1 transaction Postgres. Lỗi giữa chừng → tự động rollback. Schema không bị nửa cập nhật.

Vấn đề: Migration đã commit một phần (vd: ALTER TABLE thành công, INSERT data fail) → cần manual rollback bằng SQL ngược lại.

Best practice em làm: Mỗi migration chỉ làm 1 việc. Tách migration phức tạp thành nhiều migration nhỏ.

J.6 — Database mất kết nối giữa transaction

Cơ chế: Postgres tự rollback transaction khi connection drop. Prisma client throw `PrismaClientKnownRequestError`. Service catch và trả 500.

Cải tiến: Retry với exponential backoff cho transaction quan trọng (chưa làm).

J.7 — Concurrent upload Excel: 2 user upload cùng tệp

Cơ chế: Multer lưu file với tên unique (`<timestamp>-<originalName>`). Hai upload không ghi đè nhau. Sau khi import xong, file tạm có thể xoá.

J.8 — File PDF quyết định upload xong nhưng DB update fail

Cơ chế: Em đã chú ý — flow là (1) upload file vào tmp, (2) mở DB transaction, (3) ghi DB, (4) move file vào thư mục chính, (5) commit DB. Nếu DB fail → file tmp bị orphan → cron cleanup mỗi giờ.

Hạn chế: Còn rủi ro orphan trong khoảng giữa step 4 và 5. Cải tiến: dùng outbox pattern.

J.9 — Validation BE bypass nếu attacker gọi trực tiếp API?

Trả lời: Joi validate ở middleware, chạy TRƯỚC controller. Bypass FE không bypass được BE.

```
router.post('/', verifyToken, validate(schema), controller.create);
//                                ^ chạy trước controller.create
```

Nếu schema validate fail → response 400 ngay, controller không được gọi.

J.10 — User nhập SQL injection vào search box?

Trả lời: Prisma parameterize `contains: input` → input bị escape, không thực thi như SQL. Vd: `' OR 1=1 --` chỉ là chuỗi tìm kiếm, không thoát query.

J.11 — Tiếng Việt có dấu trong URL?

Trả lời: URL được encode bằng `encodeURIComponent` ở FE (vd: `/personnel?keyword=Nguy%E1%BB%85n`). Express tự decode `req.query.keyword` về `'Nguyễn'`. Prisma xử lý Unicode đúng.

J.12 — Time zone — server và client khác nhau

Hiện tại: PostgreSQL lưu `Timestamp(0)` không kèm timezone. Server giả định Asia/Ho_Chi_Minh (UTC+7).

Vấn đề: Nếu server đặt ở múi khác → sai. Em set `TZ=Asia/Ho_Chi_Minh` trong `ecosystem.config.js` để đảm bảo nhất quán.

Cải tiến tương lai: Đổi sang `Timestamptz` (with time zone) để rõ ràng.

J.13 — Truy vấn 1000 record về client cùng lúc

Cơ chế chống: - `MAX_LIMIT = 100` trong `helpers/paginationHelper.ts` → service tự cap `limit` xuống 100. - Joi schema validate `limit: Joi.number().max(100)`.

J.14 — User download file không có quyền

Cơ chế: Endpoint `GET /api/decisions/download/:soQuyetDinh` đi qua `verifyToken`. USER không có ownership → service trả 403.

```
async downloadDecision(soQuyetDinh, user) {
  const decision = await fileQuyetDinhRepository.findBySoQuyetDinh(soQuyetDinh);
  if (!decision) throw new NotFoundError();
  if (user.role === 'USER' && !await isMyOwnDecision(user.quan_nhan_id, decision)) {
    throw new ForbiddenError('Bạn không có quyền tải file này');
  }
  // serve file
}
```

J.15 — User edit URL `?personnelId=` để xem đơn vị khác

Cơ chế: Manager chỉ thấy quân nhân trong cây đơn vị mình. Service luôn lọc:

```
const where = { id: req.params.id };
if (req.user.role === 'MANAGER') {
  where.id = { in: req.unitFilter.personnelIds };  // intersect với cây đơn vị
}
const personnel = await prisma.quanNhan.findFirst({ where });
if (!personnel) throw new NotFoundError();
```

J.16 — Race khi tạo `so_quyet_dinh` trùng

Cơ chế: Cột `FileQuyetDinh.so_quyet_dinh` có `@unique`. INSERT trùng → P2002 → catch và trả 409 + gợi ý số tiếp theo.

K. Câu hỏi nghiệp vụ quân đội

K.1 — Tại sao 7 loại khen thưởng mà không gộp?

Trả lời: Mỗi loại có: - Quy tắc xét khác nhau (chuỗi vs thời gian phục vụ vs chức vụ vs NCKH). - Mẫu quyết định khác nhau. - Cấp duyệt khác nhau (đơn vị, học viện, BQP). - Yêu cầu input khác (vd: KNC chỉ có 1 lần, HCCSVV có 3 hạng).

Gộp 7 loại thành 1 abstract sẽ tạo ra rule phức tạp hơn và mất tính linh hoạt khi 1 loại đổi rule.

K.2 — Tại sao BKTTCP cá nhân lifetime mà đơn vị thì không?

Trả lời: Đơn vị có thể nhận BKTTCP nhiều lần qua các chu kỳ 7 năm vì đơn vị tồn tại lâu, có thể "tái xuất sắc". Cá nhân quân nhân — theo luật hiện hành — chỉ nhận BKTTCP một lần trong sự nghiệp.

Nếu thay đổi rule: Đổi `isLifetime: false` trong `PERSONAL_CHAIN_AWARDS[BKTTCP]` là xong. Logic core chấp nhận cả hai mode.

K.3 — Hệ thống có hỗ trợ Anh hùng LLVT, Anh hùng Lao động không?

Trả lời: Chưa. Đây là tier cao hơn BKTTCP, em đã đề xuất ở §6.2 hướng (i). Mở rộng dễ vì dùng `ChainAwardConfig`.

Code mock cho buổi bảo vệ:

```
const ANH_HUNG_LLVT: ChainAwardConfig = {
  code: 'ANH_HUNG_LLVT',
  cycleYears: 0,  // không cycle
  isLifetime: true,
  prerequisites: [{ code: 'BKTTCP', count: 1 }],
  requiresNCKH: true,
  // ... thêm tiêu chí phức tạp khác
};
```

K.4 — Cấp Bộ Quốc phòng vs cấp Học viện duyệt — phân biệt thế nào?

Trả lời: - Trong project, mọi đề xuất đều do Phòng Chính trị Học viện duyệt thông qua vai trò ADMIN. - Việc gửi lên cấp BQP/Tổng cục là quy trình ngoài hệ thống (gửi văn bản giấy). - Em chưa có module tương tác với BQP — đề xuất ở Chương 6 hướng (vi) tích hợp SSO + dashboard cấp Bộ.

K.5 — Bảo mật khi LAN bị xâm nhập

Trả lời: Project chạy LAN nội bộ Học viện, không expose Internet. Nếu LAN bị xâm nhập: - Attacker vẫn cần valid credentials để đăng nhập. - Audit log ghi mọi thao tác → forensic được. - Backup chu kỳ → restore được.

Hạn chế: Nếu attacker có access DB trực tiếp (qua psql), họ thấy được password hash bcrypt → có thể brute (chậm vì cost 10).

K.6 — Có hỗ trợ kiểm tra trùng tên đơn vị không?

Trả lời: Cột `ma_don_vi` `@unique` chống trùng theo mã. Tên đơn vị không unique để cho phép 2 đơn vị khác nhau có tên giống (vd: hai “Tiểu đoàn 1”).

L. Khi không biết câu trả lời

L.1 — Khi bị hỏi về rule pháp luật cụ thể em không biết

Trả lời mẫu: “Cảm ơn thầy/cô. Phần rule pháp luật cụ thể em chưa nắm sâu — em đã căn cứ vào đề tài cấp Học viện do Thượng tá ThS. Đặng Quốc Hưng và Trung tá ThS. Bùi Đình Thế chủ trì [23] và Luật Thi đua Khen thưởng số 06/2022/QH15 [1]. Em sẽ ghi nhận để bổ sung.”

L.2 — Khi bị hỏi công nghệ em chưa làm

Trả lời mẫu: “Em chưa thực hành phần đó trong đồ án. Theo em hiểu thì [trả lời lý thuyết]. Em sẽ thử trong dự án thực tế.”

L.3 — Khi bị bắt sai logic / bug

Trả lời mẫu: “Vâng, đúng là chỗ đó em xử lý chưa hết. Em sẽ ghi vào danh sách hạn chế và phát triển thêm. Cảm ơn thầy/cô đã chỉ ra.”

Tuyệt đối tránh: cãi tay đôi, đổ lỗi cho thư viện hoặc người khác.

L.4 — Khi bị hỏi “tại sao không dùng X mới hơn?”

Trả lời mẫu: “Thời điểm em chọn công nghệ (đầu 2025), X chưa đủ chín hoặc cộng đồng còn nhỏ. Em ưu tiên ổn định và hỗ trợ lâu dài. Trong tương lai, khi X stable hơn, em sẽ cân nhắc migrate.”

L.5 — Khi bị hỏi về số liệu cụ thể em không nhớ

Trả lời mẫu: “Em chưa nhớ chính xác con số đó. Theo em ước tính khoảng [X]. Nếu cần em có thể chạy lại benchmark trên máy demo.”

L.6 — Khi câu hỏi quá rộng / quá khó

Trả lời mẫu: “Câu hỏi này khá rộng. Em xin trả lời trong phạm vi project — [trả lời phần em biết]. Phần [phần khác] vượt ngoài phạm vi đồ án, em sẽ tìm hiểu thêm.”

M. Khả năng bảo trì và mở rộng kiến trúc

M.1 — Tại sao kiến trúc của em “dễ bảo trì”? Đo bằng tiêu chí gì?

Ngắn: Em đánh giá maintainability theo 5 chỉ số định lượng: (1) tách lớp rõ qua quy ước thư mục, (2) phụ thuộc đơn hướng giữa các lớp, (3) file ≤ 500 LOC, (4) test coverage ≥ 85 % cho service trọng yếu, (5) tài liệu CLAUDE.md cho mọi quy ước. Cả 5 đều có số đo cụ thể trong project.

Chi tiết:

Tiêu chí	Đo trong project	Cách verify
Tách lớp rõ	6 lớp Route → Middleware → Controller → Service → Repository → Prisma; mỗi lớp 1 thư mục riêng	`ls BE-QLKT/src/` thấy 6 folder tương ứng
Phụ thuộc đơn hướng	Controller chỉ import service, service chỉ import repository, repository chỉ import prisma	`grep "from '../models'"` trong `controllers/` ra 0 kết quả (anti-pattern AP-1)
File ≤ 500 LOC	95 % file đạt; file vượt được tách (vd: `proposal/approve.ts` từ 2001 LOC → 480 LOC + 4 sub-file)	`find src/services -name "*.ts" -exec wc -l {} \;`
Test coverage ≥ 85 %	`services/profile`, `services/eligibility`, `services/proposal` đạt; controller ~60 %	`npm run test:coverage`
Tài liệu quy ước	Root `CLAUDE.md` + `BE-QLKT/CLAUDE.md` + `FE-QLKT/CLAUDE.md` ghi rõ naming convention, anti-pattern, pattern bắt buộc	Mở 3 file đó

Phản biện: "5 chỉ số đó có chuẩn công nghiệp nào không?" → "Em tham khảo từ Clean Code (Robert Martin) và bộ chỉ số Sonar (cyclomatic complexity, code duplication, nesting depth) — đã ghi vào `BE-QLKT/CLAUDE.md` thành 9 anti-pattern bắt buộc tránh."

M.2 — Thêm 1 loại khen thưởng mới (ví dụ "Anh hùng LLVT") cần sửa bao nhiêu file?

Trả lời ngắn: 5–7 file. Quy trình rõ ràng.

Quy trình từng bước:

```
1. constants/danhHieu.constants.ts
   → thêm enum 'ANH_HUNG_LLVT' vào DANH_HIEU_CA_NHAN

2. constants/chainAwards.constants.ts
   → thêm vào PERSONAL_CHAIN_AWARDS:
     {
       code: 'ANH_HUNG_LLVT',
       cycleYears: 0,                    // không cycle
       isLifetime: true,                 // 1 lần duy nhất
       prerequisites: [{ code: 'BKTTCP', count: 1 }],
       requiresNCKH: true,
     }

3. prisma/schema.prisma
   → thêm cột nhan_anh_hung_llvt + so_quyet_dinh_anh_hung_llvt vào DanhHieuHangNam
   → npx prisma migrate dev --name add_anh_hung_llvt

4. services/proposal/strategies/anhHungLlvtStrategy.ts (file mới)
   → implement ProposalStrategy interface (4 method)

5. services/proposal/strategies/index.ts
   → thêm 1 dòng vào REGISTRY

6. validations/proposal.validation.ts
   → thêm Joi schema cho loại mới

7. tests/services/eligibility-anh-hung-llvt-personal.test.ts (file mới)
   → 30-50 ca kiểm thử
```

Không cần sửa: route file (proposal route generic), controller (dispatch qua REGISTRY), audit log (helper generic), notification (helper generic), FE form (UI generate từ schema).

Đây là minh chứng mạnh nhất cho extensibility — em sẵn sàng demo live nếu hội đồng yêu cầu.

M.3 — Thêm 1 vai trò mới (ví dụ "ANALYST" — chỉ đọc dashboard)

Quy trình:

```
1. constants/roles.constants.ts
   → thêm ROLES.ANALYST = 'ANALYST'

2. middlewares/auth.ts
   → thêm const requireAnalyst = checkRole([ROLES.SUPER_ADMIN, ROLES.ANALYST])

3. routes/dashboard.route.ts
   → đổi requireAdmin thành checkRole([ROLES.ADMIN, ROLES.ANALYST])

4. seed script hoặc account create endpoint
   → cho phép tạo tài khoản ANALYST
```

Không cần sửa logic eligibility, proposal, personnel.

M.4 — Thêm 1 endpoint mới (vd: “Lấy thống kê theo quý”)

Quy trình theo BE-QLKT/CLAUDE.md mục “Adding a New Feature”:

```
1. validations/dashboard.validation.ts
   → schema query (year + quarter)

2. services/dashboard.service.ts
   → method getQuarterlyStats(year, quarter)

3. controllers/dashboard.controller.ts
   → catchAsync wrap, gọi service, trả ResponseHelper.success

4. routes/dashboard.route.ts
   → router.get('/quarterly', verifyToken, requireAdmin, validate(...), auditLog(...),
controller.getQuarterlyStats)
```

Total ~ 4 file, 30 phút công việc nếu logic không phức tạp.

M.5 — Service A có 1500 dòng — em xử lý sao để dễ bảo trì?

Quy tắc trong project (`BE-QLKT/CLAUDE.md`):

LOC	Hành động
< 500	Để nguyên
500–800	Cân nhắc tách
800–1000	Bắt buộc tách logic phức tạp ra helper
> 1000	**Bắt buộc** tách theo pattern: `<feature>.ts` orchestration mỏng + `<feature>/types.ts` + `<feature>/<concern>.ts` cho mỗi concern

Ví dụ thực tế — `services/proposal/approve.ts` từng có 2001 LOC:

```
TRƯỚC:
services/proposal/approve.ts (2001 LOC) — chứa hết validation, mapping, import
dispatch

SAU:
services/proposal/approve.ts (480 LOC) — orchestration: parse → validate → map →
import → log
services/proposal/approve/types.ts (50 LOC) — shared interface
services/proposal/approve/validation.ts (380 LOC) — pre-flight check
services/proposal/approve/decisionMappings.ts (240 LOC) — decision metadata + PDF
persist
services/proposal/approve/import.ts (320 LOC) — transaction import dispatch
```

Mỗi file giờ đảm nhận 1 concern, dễ test riêng, dễ tìm khi debug.

M.6 — Strategy pattern khi nào nên dùng? Khi nào overengineering?

Dùng khi: - ≥ 4 nhánh `if/else` dispatch theo enum/type. - Mỗi nhánh có ≥ 2 method tương tự cấu trúc. - Có khả năng thêm nhánh mới trong tương lai.

Đừng dùng khi: - 2-3 nhánh đơn giản → `if/else` rõ ràng hơn. - Mỗi nhánh chỉ 1 dòng → switch statement gọn hơn. - Không có khả năng mở rộng → tạo interface chỉ để có 1 implementation là overengineering.

Trong project em dùng Strategy ở 2 chỗ: 1. `services/proposal/strategies/` — 7 loại đề xuất, mỗi loại có 4 method. Đáng dùng. 2. **(KHÔNG dùng)** cho `notification/` — chỉ 3 loại notification, dispatch đơn giản, để `if/else`.

M.7 — Đặt tên biến/file/function — quy ước thế nào?

Bảng quy ước (trong `CLAUDE.md`):

Loại	Pattern	Ví dụ
React component	PascalCase.tsx	`LoginForm.tsx`
Hook	camelCase.ts	`useFetch.ts`
BE service	camelCase.service.ts	`proposal.service.ts`
BE controller	camelCase.controller.ts	`account.controller.ts`
BE route	camelCase.route.ts	`auth.route.ts`
Constants file	camelCase.constants.ts	`roles.constants.ts`
Constant value	UPPER_SNAKE_CASE + `as const`	`ROLES.SUPER_ADMIN`
Function/var	camelCase	`computeChainContext`
Prisma model	PascalCase Vietnamese + `@@map("snake_case")`	`model QuanNhan { ... @@map("QuanNhan") }`
DB field	snake_case Vietnamese	`ho_ten`, `ngay_sinh`, `quan_nhan_id`
Type/Interface	PascalCase	`ApiResponse<T>`, `ProposalStrategy`

Lý do: Convention nhất quán → grep nhanh, IDE autocomplete chính xác, code review giảm tranh cãi.

M.8 — Khi nào extract helper, khi nào để inline?

Extract khi: - Logic lặp lại 2+ lần (DRY). - Logic phức tạp ≥ 10 dòng có thể test riêng. - Pure function (không side effect).

Để inline khi: - Dùng 1 lần và chỉ 3-5 dòng. - Phụ thuộc vào nhiều biến local context.

Quy tắc helper (`AP-3`): Helper trong `helpers/` **không được** import `prisma` , `apiClient` , hay service. Chỉ pure function. Exception: `auditLog/` và `notification/` vốn coupling với DB.

M.9 — Database schema thay đổi — workflow đảm bảo không lệch giữa môi trường?

Workflow chuẩn:

```
1. Sửa prisma/schema.prisma trên máy dev
2. npx prisma migrate dev --name <tên_migration>
   → Sinh file migration SQL trong prisma/migrations/<timestamp>_<tên>/
3. Test trên dev xong → commit cả schema.prisma + thư mục migration
4. Trên staging/production: npx prisma migrate deploy (chỉ apply, không sinh migration
mới)
5. Verify: npx prisma migrate status
```

Schema diff luôn versioned trong git → bất kỳ ai checkout code cũng có cùng schema.

Quy tắc nguy hiểm (AP-8): Cột có data → KHÔNG dùng `prisma db push` để rename. Phải viết script raw SQL `ALTER TABLE ... RENAME COLUMN ...` trong `src/scripts/` , chạy script trước, rồi mới `db push` để sync.

M.10 — Mở rộng sang microservice trong tương lai — kiến trúc hiện tại có cản trở không?

Trả lời: Layered architecture hiện tại **đã sẵn sàng tách microservice** vì các lớp coupling lỏng:

Module	Tách thành microservice riêng	Mức độ khó
`services/auth.service.ts` + `routes/auth.route.ts`	**Auth Service**	Dễ — đã decoupled
`services/eligibility/` + `services/profile/`	**Eligibility Engine** (gRPC)	Trung bình — phụ thuộc DB schema chuỗi
`services/notification/` + `utils/socketService.ts`	**Notification Service**	Dễ — message queue (Redis/Kafka)
`services/backup.service.ts`	**Backup Worker**	Dễ — cron riêng
`services/proposal/`	Để ở core	Phức tạp do liên kết nhiều bảng

Hạn chế hiện tại: Tất cả service share 1 database PostgreSQL → tách microservice phải tách DB (database-per-service), kèm theo distributed transaction (saga pattern). Đây là công việc lớn — em đã ghi vào hướng phát triển.

M.11 — Scale ngang (horizontal scaling) thế nào?

Cấu hình hiện tại: 1 server LAN, ~50 user concurrent.

Khi cần scale: 1. **PM2 cluster mode:** `pm2 start ecosystem.config.js -i max` → 1 process / CPU core. 4 cores = 4 instance trên cùng máy. 2. **Phân tách stateful state:** - `refreshToken` đang lưu DB → vẫn share giữa instance. - Socket.IO room — cần Redis adapter (`@socket.io/redis-adapter`) khi nhiều process. 3. **Load balancer:** Nginx upstream round-robin giữa các process. 4. **Database read replica:** Tách query đọc sang replica, ghi vào primary. Prisma support qua `replicaUrls`. 5. **Migration sang Kubernetes:** Khi vượt 1 server vật lý.

Bottleneck dự kiến đầu tiên: Database write throughput. Khi đó dùng partitioning theo năm cho `DanhHieuHangNam` (table lớn nhất khi tích luỹ nhiều năm).

M.12 — Module split theo ngữ cảnh — nguyên tắc?

Nguyên tắc 1 helper file = 1 responsibility: - `helpers/excelImportHelper.ts` chỉ import (đọc workbook, validate dòng). - `helpers/excelTemplateHelper.ts` chỉ template (sinh workbook mẫu). - KHÔNG có file `helpers/excelHelper.ts` chứa cả hai.

Quy tắc service: - 1 service = 1 entity (vd: `personnel.service.ts`, `proposal.service.ts`). - Nếu 3+ service có logic giống nhau → extract vào `services/<feature>/shared.ts` hoặc `helpers/<feature>/`.

Quy tắc route: - 1 route file = 1 prefix (vd: `auth.route.ts` cho `/api/auth`). - Tránh route file > 200 LOC bằng cách tách subresource ra file riêng (vd: `personnelNested.route.ts` cho `/api/personnel/:id/...`).

M.13 — TypeScript strict mode

Hiện tại: `strict: false`, `strictNullChecks: false` trong `tsconfig.json` — relaxed mode.

Lý do chọn relaxed: - Cho phép linh hoạt khi xử lý kết quả Prisma (nhiều null trong nested include). - Giảm boilerplate `?? null` không cần thiết.

Bù lại: - Joi validate input ở boundary → đảm bảo type runtime. - Test cases cover các trường hợp null/undefined.

Hạn chế thừa nhận: `strict: true` sẽ bắt nhiều bug hơn ở compile time. Nếu rebuild project sẽ bật strict ngay từ đầu.

M.14 — Dependency injection — em có dùng không?

Trả lời: Không dùng container DI (như NestJS/InversifyJS). Em dùng **manual wiring** qua singleton:

```
// services/proposal.service.ts
class ProposalService {
  // không inject — dùng repository singleton trực tiếp
}
export const proposalService = new ProposalService();
```

Lý do: - Project quy mô vừa (~50 service) — manual wiring vẫn quản lý được. - Container DI thêm phụ thuộc + decorator + reflect-metadata → không cần thiết. - Để mock test, em mock module qua `jest.mock('../repositories/...')`.

Khi nào nên đổi: Nếu project lên 200+ service hoặc cần DI scope (request scope, transient) thì chuyển NestJS.

M.15 — Versioning API — em xử lý sao khi cần breaking change?

Hiện tại: Tất cả endpoint dưới `/api/...` không có version. Đây là acceptable cho project nội bộ 1 client.

Khi cần versioning: - Strategy 1: URL prefix `/api/v1/...`, `/api/v2/...`. - Strategy 2: Header `X-API-Version: 2`. - Strategy 3: Field trong response: thêm `_version: 2` cho client kiểm.

Best practice: Giữ v1 song song với v2 ít nhất 6 tháng → client có thời gian migrate.

M.16 — Code review process

Hiện tại: Solo developer (1 mình em).

Cơ chế tự review: - Trước khi commit: `npm run typecheck && npm test && npm run lint`. - File `PROJECT_REVIEW.md` ghi các issue self-found theo CRITICAL/MEDIUM/LOW. - File `CLAUDE.md` định nghĩa anti-pattern bắt buộc tránh khi viết code mới.

Khi mở team: - Bắt buộc PR review qua GitHub. - CI pass mới merge. - Rule pre-commit hook chạy lint + format.

N. Tổng hợp chống tấn công và đánh giá an toàn

N.1 — Hệ thống của em có an toàn không? Đánh giá thẳng thắn.

Trả lời ngắn — KHÔNG né:

"Hệ thống của em an toàn ở mức **đủ cho môi trường LAN nội bộ Học viện**, nhưng **chưa đủ** để expose ra Internet công cộng. Em chia mức độ an toàn theo 3 cấp:

- **Đã chống tốt (đạt OWASP Top 10):** SQLi, XSS, CSRF, IDOR, BOLA, Mass Assignment, Brute force, file upload độc, Privilege escalation, Path traversal, missing auth.
- **Đã có nhưng cần cải thiện:** Rate limit (chỉ theo IP, chưa theo account), JWT rotation (chưa có grace period), audit log (chưa ghi failed login chi tiết).
- **Chưa có (thừa nhận):** 2FA, Web Application Firewall (WAF), Penetration test chính thức, Security headers nâng cao (CSP nonce), Anti-CSRF token (em né bằng JWT header), DDoS protection layer 4, Encryption at rest cho DB."

N.2 — Bảng tổng hợp 10 kiểu tấn công × cơ chế phòng

#	Kiểu tấn công (OWASP/CWE)	Cơ chế chống trong project	Tệp xử lý chính	Còn rủi ro?
1	**SQL Injection** (CWE-89)	Prisma parameterize tự động; không dùng `$queryRawUnsafe` cho input từ user	`models/index.ts` + mọi service	Không
2	**XSS** (CWE-79)	React escape mọi `{value}`; không có `dangerouslySetInnerHTML` cho user input; helmet headers	FE components + `app.ts` helmet	Thấp
3	**CSRF** (CWE-352)	JWT trong header `Authorization`, không cookie session → browser không tự gửi cross-origin	`middlewares/auth.ts`	Không
4	**IDOR / BOLA** (CWE-639)	3 lớp: `verifyToken` + `requireRole` + ownership check trong service; `unitFilter` lọc theo cây đơn vị cho MANAGER	`auth.ts`, `unitFilter.ts`, services	Thấp
5	**Brute force password** (CWE-307)	`authLimiter` 10 req / 15 phút / IP; bcrypt cost 10 (~100 ms/lần thử)	`configs/rateLimiter.ts`	Trung bình (chưa account lockout)
6	**Mass Assignment** (CWE-915)	Joi `stripUnknown: true` ở mọi endpoint; service không truyền `req.body` thẳng vào `prisma.create`	`middlewares/validate.ts` + `validations/`	Không
7	**File upload độc** (CWE-434)	Multer whitelist extension + MIME + size 10 MB; lưu ngoài web root; check magic byte cho PDF	`configs/multer.ts`	Thấp (chưa scan virus)
8	**Privilege escalation** (CWE-269)	Role trong JWT chữ ký HMAC, không sửa được client-side; Joi schema không cho update field `role` qua self-update	JWT + Joi	Không
9	**Path traversal** (CWE-22)	File path từ DB chứ không từ user; `path.basename` strip mọi `../`	`decision.service.ts` download	Không

#	Kiểu tấn công (OWASP/CWE)	Cơ chế chống trong project	Tệp xử lý chính	Còn rủi ro?
10	**Missing auth** (CWE-306)	`verifyToken` middleware bắt buộc trước mọi route nghiệp vụ; không có endpoint nghiệp vụ public	Mọi `routes/*.ts`	Không
11	**Sensitive Data Exposure** (CWE-200)	Prisma `select` whitelist field; không trả `password_hash`, `refreshToken`; CCCD ẩn cho USER	services + helpers	Thấp
12	**Insecure Deserialization** (CWE-502)	Không có deserialize từ user input (không dùng `JSON.parse` lên payload nhạy cảm); JWT verify chữ ký trước khi đọc	`auth.ts`	Không
13	**DoS / Resource exhaustion** (CWE-400)	Rate limit, body limit 10 MB, MAX_LIMIT 100 records, file size 10 MB	rateLimiter, paginationHelper	Trung bình (DDoS layer 4 không chống)
14	**Clickjacking** (CWE-1021)	helmet `X-Frame-Options: DENY`	`app.ts`	Không
15	**Insufficient logging** (CWE-778)	Audit log mọi mutate, ghi `actor_role`, `payload`, `ip_address`, `user_agent`	`middlewares/auditLog.ts`	Thấp (chưa log failed auth)

N.3 — Tự tay tấn công hệ thống — em đã thử những gì?

Em đã tự pentest cơ bản:

Kịch bản tấn công	Công cụ	Kết quả
SQLi vào search box “ho_ten=’ OR 1=1 –”	Postman manual	Bị Prisma escape, trả về 0 record (tìm tên = chuỗi đó)
XSS payload `<script>alert(1)</script>` vào ghi chú	UI nhập tay	Render thành text, không execute
Đăng nhập sai 11 lần	Postman lặp	Lần 11 nhận 429 Too Many Requests
USER call `GET /api/personnel/<other_id>`	Postman với JWT của USER	403 Forbidden
MANAGER call `GET /api/personnel?co_quan_don_vi_id=<other_unit>`	Postman	unitFilter lọc, trả 0 record
Tự sửa JWT đổi `role` thành ADMIN	jwt.io tool, decode + re-sign sai key	`jwt.verify` fail → 401
Upload file `.exe` đổi tên `.pdf`	Postman	MIME check fail → 400
Upload file 50 MB	Postman	Multer limit 10 MB → 413 Payload Too Large
Tạo 1000 đề xuất / 1 phút từ MANAGER	Script	writeLimiter 30 req / 15 phút → 429

Chưa thử: automated scan với Burp Suite, OWASP ZAP, sqlmap. Em đề xuất ở Chương 6 hướng phát triển.

N.4 — Threat model: ai có thể tấn công, mức độ thiệt hại?

Bảng STRIDE:

Threat	Tác nhân	Mức độ	Đã chống	Còn lại
Spoofing (giả mạo identity)	User cùng LAN biết username người khác	Cao	bcrypt + JWT signature	Brute force chậm (cost 10)
Tampering (sửa data)	Attacker có access DB	Cao	Audit log mọi mutate	Không chống được DBA
Repudiation (chối bỏ)	User chối thao tác đã làm	Trung bình	Audit log có IP + user agent	Cần legal disclaimer
Information Disclosure (rò rỉ)	User truy vấn nhầm endpoint	Trung bình	Role + unit filter	Internal LAN — không firewall ngoài
Denial of Service	User phá hệ thống bằng request	Thấp (LAN)	Rate limit, body limit	DDoS layer 4 (cần WAF)
Elevation of Privilege	USER tự nâng thành ADMIN	Cao	JWT signed + role không updateable qua self-update	Không (đã chống tốt)

N.5 — JWT secret bị lộ — quy trình rotation?

Hiện tại: `JWT_SECRET` và `JWT_REFRESH_SECRET` lưu trong file `.env`, chmod 600, owner root.

Nếu lộ: 1. **Khẩn cấp:** đổi `JWT_SECRET` mới trong `.env`, restart server (`pm2 reload`). 2. Mọi access token cũ trở thành invalid → user phải đăng nhập lại. 3. **Refresh token:** đổi `JWT_REFRESH_SECRET` riêng → buộc đăng nhập lại với username/password. 4. Audit log review để xem có thao tác bất thường trong khoảng lộ.

Cải tiến: Lưu secret trong vault (HashiCorp Vault, AWS Secrets Manager) thay vì `.env`. Hiện chưa có vì LAN cô lập.

N.6 — Database admin (DBA) có lạm quyền — chống thế nào?

Trả lời thẳng thắn:

"Em chưa có cơ chế chống DBA insider threat hoàn toàn. Người có access PostgreSQL trực tiếp (qua `psql` hoặc `pgAdmin`) có thể: - Xem tất cả data (kể cả password hash bcrypt — phải brute mới ra plaintext). - Sửa data bypass audit log (vì audit log ghi qua middleware app, không qua DB trigger). - Xóa cả audit log.

Cách giảm thiểu: - Phân tách tài khoản DB: 1 tài khoản app dùng (chỉ INSERT/UPDATE/DELETE/SELECT các bảng cụ thể), 1 tài khoản DBA tách biệt. - Database trigger ghi log song song (em chưa làm — đề xuất ở hướng phát triển). - Encrypt at rest dùng `pgcrypto` cho cột nhạy cảm (em chưa làm). - Audit ở tầng OS: `auditd` Linux ghi access vào file DB."

N.7 — Backup file bị copy ra ngoài — rủi ro?

Trả lời: - Backup `pg_dump` ở dạng SQL plain text. - Chứa **tất cả data** + password hash bcrypt. - Attacker copy được → có thể brute password offline.

Bảo vệ: - Thư mục `backups/` chmod 700, owner postgres. - Khi sao lưu sang storage ngoài (NAS), phải mã hoá bằng GPG hoặc AES-256. - Hiện chưa làm — đề xuất ở hướng phát triển.

N.8 — Audit log bị xoá bởi SUPER_ADMIN — có chống không?

Trả lời thẳng: - Có endpoint `DELETE /api/system-logs/...` chỉ SUPER_ADMIN gọi được. - SUPER_ADMIN xóa log → mất bằng chứng. - Đây là rủi ro inherent của insider threat.

Cách giảm thiểu: - Log mọi thao tác xóa log (meta-log) cũng vào bảng SystemLog → SUPER_ADMIN xóa cả meta-log thì mất luôn. - Forward log realtime sang server log riêng (Splunk, ELK) — write-only từ phía app, SUPER_ADMIN của app không có quyền sửa server log. - Hiện em chưa làm — đề xuất hướng phát triển.

N.9 — DDoS layer 4 — chống được không?

Trả lời: - Hiện không. Express + Node.js không chống được SYN flood, UDP flood ở tầng mạng. - Cần WAF (Cloudflare, AWS Shield) hoặc reverse proxy có rate-limit IP-level (Nginx + `limit_req_zone`).

Trong context project: - Chạy LAN nội bộ — DDoS phải từ trong LAN → có nghĩa là người trong tổ chức. - Nếu cần thiết: kích hoạt Nginx `limit_req_zone $binary_remote_addr zone=one:10m rate=10r/s` chặn ở edge.

N.10 — Encryption at rest — DB có mã hoá không?

Hiện tại: Không. PostgreSQL data trên disk dạng plain.

Có thể mã hoá: 1. **Disk-level:** LUKS encryption cho partition chứa `/var/lib/postgresql`. 2. **Column-level:** dùng `pgcrypto`, vd `password_hash = pgp_sym_encrypt(value, key)`. 3. **TLS giữa app và DB:** PostgreSQL hỗ trợ TLS, hiện chưa bật.

Trong project: chưa cần vì LAN cô lập, server đặt trong phòng khoá. Đề xuất khi expose ra Internet.

N.11 — Dependency vulnerabilities — em quản lý sao?

Cơ chế: - `npm audit` chạy thủ công mỗi tuần. - `npm outdated` để xem package cũ. - Khi có CVE: update minor/patch version qua `npm update`. Major version cần test cẩn thận.

Hiện tại: `npm audit` báo 0 high/critical (chỉ vài low).

Đề xuất: Dependabot hoặc Renovate bot tự tạo PR khi có CVE.

N.12 — Security headers — có set đầy đủ không?

Hiện tại qua `helmet()`:

```
import helmet from 'helmet';
app.use(helmet());
```

Helmet mặc định set: - `Strict-Transport-Security` (HSTS) - `X-Content-Type-Options: nosniff` - `X-Frame-Options: DENY` - `X-XSS-Protection: 0` (disabled vì gây bug ở browser cũ) - `Content-Security-Policy: default-src 'self'` - `Referrer-Policy: no-referrer`

Có thể cải thiện: - CSP nonce-based thay vì `'self'` cho strict hơn. - `Permissions-Policy` chặn camera/microphone API. - HSTS preload list.

N.13 — Logging chuẩn forensic — đủ chưa?

Audit log hiện ghi: - `nguoi_thuc_hien_id`, `actor_role` - `action`, `resource`, `tai_nguyen_id` - `description` tiếng Việt - `payload` JSON (before/after) - `ip_address`, `user_agent` - `createdAt` precision second

Đủ cho: - Truy ai làm gì lúc nào. - Reconstruct dữ liệu trước khi sửa. - Identify session anomaly (vd: 1 user login từ 2 IP khác nhau trong 1 phút).

Chưa đủ cho: - Failed login attempt (chưa log fail). - Tracking session hijack (chưa correlate user agent + IP fingerprint). - Long-term retention (chưa có archive policy → audit log lớn vô hạn).

N.14 — Penetration test chính thức — có không?

Trả lời thẳng: "Em chưa thuê pentest chính thức (do chi phí và quy trình). Em chỉ tự pentest cơ bản như mô tả ở N.3. Trước khi triển khai thật cho Học viện, sẽ đề xuất nhờ Phòng Bảo mật của Tổng cục II hoặc đơn vị chuyên môn quân đội thực hiện pentest."

N.15 — Compliance / GDPR / Luật bảo vệ dữ liệu cá nhân Việt Nam

Trả lời: - Project xử lý dữ liệu cá nhân (CCCD, ngày sinh, quê quán, lịch sử công tác) — thuộc **dữ liệu cá nhân nhạy cảm** theo Nghị định 13/2023/NĐ-CP. - Đã có: - Audit log truy cập. - Phân quyền hạn chế chia sẻ. - Backup an toàn. - **Chưa có:** - Quyền xoá data theo yêu cầu cá nhân (right to erasure). - Export data theo định dạng chuẩn (right to portability). - Privacy policy hiển thị cho user. - Vì là hệ thống nội bộ quân sự, áp dụng quy chế quân đội khác với GDPR — cần tuân theo Nghị quyết của Quân uỷ Trung ương và quy chế của Học viện về bảo vệ thông tin.

N.16 — Tóm tắt đánh giá an toàn cuối cùng

Tổng kết 1 đoạn:

"Hệ thống của em đạt mức an toàn **xếp loại Khá** cho ứng dụng web nội bộ. **Đã chống được** 12/15 mục OWASP Top 10 ở mức tốt. **Còn 3 hạn chế** cần cải thiện trước khi mở rộng quy mô: (1) chưa có 2FA cho tài khoản đặc quyền cao, (2) chưa có cơ chế chống DBA insider threat, (3) chưa có pentest chính thức. Trong phạm vi project nội bộ LAN Học viện chạy ~50 user, mức độ an toàn này **đủ dùng** và **không có lỗ hổng cấp critical** mà em phát hiện được. Em đã ghi 3 hạn chế trên vào hướng phát triển ở Chương 6."

N.17 — Câu hỏi 2FA — tại sao chưa làm?

Trả lời: - 2FA cho LAN nội bộ ít cần thiết hơn cho Internet — vì đã có lớp bảo vệ vật lý (vào phòng máy phải qua kiểm tra). - Triển khai 2FA cần infrastructure: SMS gateway (quân đội có riêng), TOTP app (Google Authenticator), hoặc hardware token (YubiKey). - Em đề xuất 2FA **chỉ cho SUPER_ADMIN và ADMIN** ở phiên bản kế tiếp.

Code stub:

```
// Tương lai: thêm cột totp_secret vào TaiKhoan
// Login flow: username + password → check totp_code (otplib) → cấp JWT
```

N.18 — Khi attacker đã vào được system rồi — incident response

Quy trình em đề xuất (chưa formalize):

1. **Detection:** Audit log alert khi có pattern bất thường (vd: 1 ADMIN xoá > 100 record / phút).
2. **Containment:**
   - Vô hiệu tài khoản nghi ngờ qua endpoint admin.
   - Đổi mọi `JWT_SECRET` → buộc tất cả user đăng nhập lại.
3. **Eradication:** Restore DB từ backup gần nhất trước thời điểm bị xâm nhập.
4. **Recovery:** Verify integrity → khởi động lại dịch vụ.
5. **Post-mortem:** Phân tích audit log để hiểu vector xâm nhập, fix lỗ hổng.

Hiện tại chưa có: - Automated alerting (Slack/email khi pattern bất thường). - Runbook chi tiết. - Drill incident response định kỳ.

N.19 — Câu chốt khi bị truy vấn nặng về bảo mật

Trả lời mẫu cho hội đồng:

"Em hiểu rằng bảo mật là quá trình liên tục, không có hệ thống nào tuyệt đối an toàn. Trong phạm vi đồ án sinh viên với 6 tháng thực hiện, em đã ưu tiên (1) chống các lỗ hổng OWASP Top 10 phổ biến nhất, (2) áp dụng nguyên tắc đặc quyền tối thiểu qua phân quyền 4 cấp, (3) ghi audit log đầy đủ để truy hồi khi sự cố. Những phần còn thiếu như 2FA, encryption at rest, pentest chính thức — em đã ghi vào hướng phát triển và sẽ triển khai khi có thêm nhân lực hỗ trợ. Em xin tiếp thu mọi góp ý của thầy/cô để cải thiện thêm."

O. Truy vấn nâng cao và tối ưu cơ sở dữ liệu

> Phần này tập trung vào những truy vấn thực tế phức tạp mà hội đồng giỏi DB có thể yêu cầu viết tại chỗ. Mỗi câu có (1) mô tả nghiệp vụ, (2) Prisma cách làm, (3) SQL tương đương, (4) giải thích kỹ thuật và (5) cách tối ưu.

O.1 — Cây đơn vị nhiều cấp: tìm tất cả đơn vị con cháu (recursive)

Nghiệp vụ: Một CQDV có nhiều DVTT con; cần lấy danh sách tất cả quân nhân thuộc cây con của 1 CQDV bất kỳ. Schema hiện tại 2 cấp (CQDV → DVTT). Nếu mở rộng nhiều cấp sẽ cần recursive CTE.

SQL recursive CTE:

```sql
WITH RECURSIVE don_vi_tree AS (
  SELECT id, parent_id, ten_don_vi, 0 AS depth
  FROM "DonVi" WHERE id = $1

  UNION ALL

  SELECT dv.id, dv.parent_id, dv.ten_don_vi, t.depth + 1
  FROM "DonVi" dv
  JOIN don_vi_tree t ON dv.parent_id = t.id
  WHERE t.depth < 10  -- chống infinite loop
)
SELECT qn.* FROM "QuanNhan" qn
WHERE qn.don_vi_id IN (SELECT id FROM don_vi_tree);
```

Prisma không hỗ trợ recursive CTE native → fallback `$queryRaw`.

Trong project hiện tại: vì cây 2 cấp em dùng cách đơn giản ở `unitFilter.ts`:

```sql
const dvttIds = await donViTrucThuocRepository.findIdsByCoQuanDonViId(cqdvId);
const personnel = await prisma.quanNhan.findMany({
  where: { OR: [{ co_quan_don_vi_id: cqdvId }, { don_vi_truc_thuoc_id: { in: dvttIds }
        }] },
});
```

Phản biện: “Sao không dùng nested set / materialized path?” → “Cây 2 cấp chưa cần. Khi mở > 3 cấp em chuyển materialized path (`path: '/cqdv1/dvtt2/td3'`) để query 1 phát.”

O.2 — Top N per group: 3 danh hiệu mới nhất của mỗi quân nhân

SQL với `ROW_NUMBER`:

```sql
WITH ranked AS (
  SELECT *,
    ROW_NUMBER() OVER (PARTITION BY quan_nhan_id ORDER BY nam DESC) AS rn
  FROM "DanhHieuHangNam"
)
SELECT * FROM ranked WHERE rn <= 3;
```

SQL alternative `LATERAL JOIN`:

```sql
SELECT qn.id, dh.*
FROM "QuanNhan" qn
LEFT JOIN LATERAL (
  SELECT * FROM "DanhHieuHangNam" dh
  WHERE dh.quan_nhan_id = qn.id
  ORDER BY nam DESC
  LIMIT 3
) dh ON TRUE;
```

Prisma: Không hỗ trợ window/lateral → `$queryRaw`. Hoặc workaround query toàn bộ rồi filter trong code (chỉ chấp nhận khi dataset nhỏ).

O.3 — Tìm chuỗi CSTĐCS dài nhất liên tục — gaps and islands

SQL:

```sql
WITH numbered AS (
  SELECT quan_nhan_id, nam,
    nam - ROW_NUMBER() OVER (PARTITION BY quan_nhan_id ORDER BY nam) AS grp
  FROM "DanhHieuHangNam"
  WHERE danh_hieu = 'CSTDCS'
),
streaks AS (
  SELECT quan_nhan_id, grp, COUNT(*) AS streak_length,
    MIN(nam) AS streak_start, MAX(nam) AS streak_end
  FROM numbered
  GROUP BY quan_nhan_id, grp
)
SELECT quan_nhan_id, MAX(streak_length) AS longest_streak
FROM streaks
GROUP BY quan_nhan_id;
```

Giải thích "gaps and islands": Nếu các năm liên tục thì hiệu giữa `nam` và `ROW_NUMBER` không đổi. Khi có gap, hiệu thay đổi → group theo hiệu.

Trong project: Em không tính qua SQL trực tiếp mà dùng JavaScript trong `lastFlagYearInChain` của `services/profile/annual.ts` — đơn giản, dễ test, đủ nhanh cho dataset 1 quân nhân.

O.4 — Tìm gaps: năm nào quân nhân X bị đứt CSTĐCS

SQL với `generate_series`:

```sql
WITH year_range AS (SELECT generate_series(2015, 2024) AS nam),
qn_dh AS (
  SELECT nam FROM "DanhHieuHangNam"
  WHERE quan_nhan_id = $1 AND danh_hieu = 'CSTDCS'
)
SELECT yr.nam AS missing_year
FROM year_range yr
LEFT JOIN qn_dh ON yr.nam = qn_dh.nam
WHERE qn_dh.nam IS NULL
ORDER BY yr.nam;
```

O.5 — Pagination tối ưu: tổng count + data trong 1 query

SQL với `COUNT(*) OVER()`:

```sql
SELECT *, COUNT(*) OVER() AS total_count
FROM "QuanNhan"
WHERE co_quan_don_vi_id = $1
ORDER BY ho_ten
LIMIT 50 OFFSET 100;
```

Mỗi row trả về kèm `total_count` (giống nhau ở mọi row). Tiết kiệm 1 round-trip DB so với `Promise.all([count, findMany])`.

Trade-off: Phức tạp hơn, mất type-safe Prisma → em vẫn giữ 2 query với `Promise.all` cho code rõ ràng.

O.6 — Anti-join: quân nhân chưa có đề xuất nào trong 3 năm

SQL với `NOT EXISTS`:

```sql
SELECT qn.* FROM "QuanNhan" qn
WHERE NOT EXISTS (
  SELECT 1 FROM "BangDeXuat" bdx
  WHERE bdx.nguoi_de_xuat_id IN (
    SELECT id FROM "TaiKhoan" WHERE quan_nhan_id = qn.id
  )
  AND bdx.nam >= EXTRACT(YEAR FROM CURRENT_DATE) - 3
);
```

Prisma `none`:

```typescript
const forgotten = await prisma.quanNhan.findMany({
  where: {
    TaiKhoan: {
      DeXuatDaGui: { none: { nam: { gte: new Date().getFullYear() - 3 } } },
    },
  },
});
```

O.7 — Conditional aggregation với FILTER

Nghiệp vụ: Dashboard — đếm pending/approved/rejected theo từng loại trong 1 query.

SQL với `FILTER`:

```
SELECT
  loai_de_xuat,
  COUNT(*) FILTER (WHERE status = 'PENDING') AS pending,
  COUNT(*) FILTER (WHERE status = 'APPROVED') AS approved,
  COUNT(*) FILTER (WHERE status = 'REJECTED') AS rejected,
  COUNT(*) AS total
FROM "BangDeXuat"
WHERE nam = $1
GROUP BY loai_de_xuat;
```

`FILTER` là syntax SQL chuẩn, gọn hơn `SUM(CASE WHEN ... THEN 1 ELSE 0 END)`.

O.8 — Pivot: chuyển hàng thành cột (số CSTĐCS theo đơn vị × năm)

SQL với crosstab (cần extension `tablefunc`):

```
CREATE EXTENSION IF NOT EXISTS tablefunc;

SELECT * FROM crosstab(
  $$
    SELECT cqdv.ten_don_vi, dh.nam, COUNT(*)
    FROM "DanhHieuHangNam" dh
    JOIN "QuanNhan" qn ON qn.id = dh.quan_nhan_id
    JOIN "CoQuanDonVi" cqdv ON cqdv.id = qn.co_quan_don_vi_id
    WHERE dh.danh_hieu = 'CSTDCS' AND dh.nam BETWEEN 2020 AND 2024
    GROUP BY cqdv.ten_don_vi, dh.nam
    ORDER BY 1, 2
  $$,
  $$ SELECT generate_series(2020, 2024) $$
) AS ct(ten_don_vi TEXT, "2020" BIGINT, "2021" BIGINT, "2022" BIGINT, "2023" BIGINT,
        "2024" BIGINT);
```

Cách đơn giản hơn: truy vấn long-format rồi pivot trong code TypeScript.

O.9 — DISTINCT ON: lấy bản ghi mới nhất của mỗi nhóm

SQL `DISTINCT ON` (PostgreSQL-specific):

```
SELECT DISTINCT ON (nguoi_thuc_hien_id) *
FROM "SystemLog"
WHERE action = 'LOGIN'
ORDER BY nguoi_thuc_hien_id, "createdAt" DESC;
```

SQL chuẩn portable:

```sql
SELECT * FROM "SystemLog" l1
WHERE action = 'LOGIN'
  AND "createdAt" = (
    SELECT MAX("createdAt") FROM "SystemLog" l2
    WHERE l2.nguoi_thuc_hien_id = l1.nguoi_thuc_hien_id AND l2.action = 'LOGIN'
  );
```

`DISTINCT ON` nhanh hơn ~2-3 lần nếu có index `(nguoi_thuc_hien_id, createdAt DESC)`.

O.10 — Cumulative sum: tổng đề xuất tích luỹ theo tháng

SQL với `SUM() OVER (ORDER BY)`:

```sql
SELECT
  DATE_TRUNC('month', "createdAt") AS thang,
  COUNT(*) AS so_de_xuat_thang,
  SUM(COUNT(*)) OVER (ORDER BY DATE_TRUNC('month', "createdAt")) AS tich_luy
FROM "BangDeXuat"
WHERE EXTRACT(YEAR FROM "createdAt") = 2025
GROUP BY DATE_TRUNC('month', "createdAt")
ORDER BY thang;
```

Kết hợp aggregation + window function — `SUM(COUNT(*))` chạy sau GROUP BY.

O.11 — Median với PERCENTILE_CONT

SQL:

```sql
SELECT
  PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY EXTRACT(YEAR FROM AGE(NOW(),
        ngay_nhap_ngu)))
    AS so_nam_phuc_vu_trung_vi
FROM "QuanNhan"
WHERE ngay_nhap_ngu IS NOT NULL;
```

`PERCENTILE_CONT(0.5)` = median; `(0.25)` = quartile 1.

O.12 — Tìm bản ghi trùng lặp (duplicate detection)

SQL:

```sql
SELECT ho_ten, ngay_sinh, COUNT(*) AS so_lan_trung, ARRAY_AGG(id) AS cac_id
FROM "QuanNhan"
WHERE ho_ten IS NOT NULL AND ngay_sinh IS NOT NULL
GROUP BY ho_ten, ngay_sinh
HAVING COUNT(*) > 1;
```

`ARRAY_AGG(id)` gom các id trùng vào 1 mảng → admin reconcile.

O.13 — Quân nhân sắp đến mốc niên hạn 10/15/20 năm trong 6 tháng

SQL:

```
SELECT qn.id, qn.ho_ten, qn.ngay_nhap_ngu,
  EXTRACT(YEAR FROM AGE(NOW(), ngay_nhap_ngu)) AS so_nam_phuc_vu,
  CASE
    WHEN EXTRACT(YEAR FROM AGE(NOW() + INTERVAL '6 months', ngay_nhap_ngu)) = 10 THEN
        'HCCSVV_BA'
    WHEN EXTRACT(YEAR FROM AGE(NOW() + INTERVAL '6 months', ngay_nhap_ngu)) = 15 THEN
        'HCCSVV_NHI'
    WHEN EXTRACT(YEAR FROM AGE(NOW() + INTERVAL '6 months', ngay_nhap_ngu)) = 20 THEN
        'HCCSVV_NHAT'
  END AS sap_dat_moc
FROM "QuanNhan" qn
WHERE EXTRACT(YEAR FROM AGE(NOW() + INTERVAL '6 months', ngay_nhap_ngu)) IN (10, 15,
        20)
  AND NOT EXISTS (
    SELECT 1 FROM "KhenThuongHCCSVV" kt
    WHERE kt.quan_nhan_id = qn.id
  );
```

O.14 — Đề xuất pending lâu nhất, cảnh báo SLA > 7 ngày

Prisma:

```
const stale = await prisma.bangDeXuat.findMany({
  where: {
    status: 'PENDING',
    createdAt: { lt: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000) },
  },
  orderBy: { createdAt: 'asc' },
});
```

SQL kèm số ngày chờ:

```
SELECT bdx.*,
  EXTRACT(EPOCH FROM (NOW() - bdx."createdAt")) / 86400 AS so_ngay_cho
FROM "BangDeXuat" bdx
WHERE bdx.status = 'PENDING' AND bdx."createdAt" < NOW() - INTERVAL '7 days'
ORDER BY bdx."createdAt" ASC;
```

O.15 — KPI người duyệt: tỷ lệ approve, thời gian duyệt trung bình

SQL:

```sql
SELECT
  t.username AS admin,
  COUNT(*) AS tong_duyet,
  COUNT(*) FILTER (WHERE bdx.status = 'APPROVED') AS approved,
  COUNT(*) FILTER (WHERE bdx.status = 'REJECTED') AS rejected,
  ROUND(100.0 * COUNT(*) FILTER (WHERE bdx.status = 'APPROVED') / NULLIF(COUNT(*), 0),
        2)
    AS ty_le_approve_pct,
  AVG(EXTRACT(EPOCH FROM (bdx.ngay_duyet - bdx."createdAt")) / 3600) AS
        gio_duyet_trung_binh
FROM "BangDeXuat" bdx
JOIN "TaiKhoan" t ON t.id = bdx.nguoi_duyet_id
WHERE bdx.ngay_duyet IS NOT NULL
  AND bdx.ngay_duyet >= NOW() - INTERVAL '6 months'
GROUP BY t.id, t.username
ORDER BY tong_duyet DESC;
```

`NULLIF(COUNT(*), 0)` chống chia cho 0.

O.16 — Self-join: cặp danh hiệu liên tiếp 2 năm

Nghiệp vụ: Tìm cặp `(năm n, năm n+1)` mà quân nhân đạt CSTĐCS cả 2 — chuẩn bị rule BKBQP.

SQL:

```sql
SELECT a.quan_nhan_id, a.nam AS nam_dau, b.nam AS nam_sau
FROM "DanhHieuHangNam" a
INNER JOIN "DanhHieuHangNam" b
  ON a.quan_nhan_id = b.quan_nhan_id
  AND b.nam = a.nam + 1
WHERE a.danh_hieu = 'CSTDCS' AND b.danh_hieu = 'CSTDCS';
```

O.17 — JSON aggregation: lịch sử thao tác dạng JSON array

SQL với `jsonb_agg`:

```sql
SELECT
  tai_nguyen_id,
  jsonb_agg(
    jsonb_build_object(
      'action', action, 'time', "createdAt",
      'by', actor_role, 'description', description
    )
    ORDER BY "createdAt" DESC
  ) AS history
FROM "SystemLog"
WHERE resource = 'personnel' AND tai_nguyen_id = $1
GROUP BY tai_nguyen_id;
```

Trả về 1 row, cột `history` là mảng JSON đã sort.

O.18 — UNION ALL: gộp đề xuất cá nhân + đơn vị

SQL:

```
SELECT id, 'CA_NHAN' AS doi_tuong, nam, status, "createdAt"
FROM "BangDeXuat"
WHERE loai_de_xuat IN ('CA_NHAN_HANG_NAM', 'NIEN_HAN', 'CONG_HIEN', 'NCKH', 'HC_QKQT',
        'KNC')
UNION ALL
SELECT id, 'DON_VI' AS doi_tuong, nam, status, "createdAt"
FROM "BangDeXuat"
WHERE loai_de_xuat = 'DON_VI_HANG_NAM'
ORDER BY "createdAt" DESC LIMIT 50;
```

`UNION ALL` không loại trùng → rẻ hơn `UNION`.

O.19 — Full-text search với tsvector

Setup index:

```
ALTER TABLE "BangDeXuat" ADD COLUMN search_vector tsvector
  GENERATED ALWAYS AS (to_tsvector('simple', coalesce(ghi_chu, ''))) STORED;
CREATE INDEX idx_bdx_search ON "BangDeXuat" USING GIN(search_vector);
```

Query:

```
SELECT * FROM "BangDeXuat"
WHERE search_vector @@ plainto_tsquery('simple', $1)
ORDER BY ts_rank(search_vector, plainto_tsquery('simple', $1)) DESC;
```

Hiện tại: em dùng `ILIKE '%keyword%'` đơn giản — không scale khi text dài. Đề xuất khi dataset > 100k bản ghi.

O.20 — Khoảng thời gian phức tạp: quý hiện tại, 12 tháng trượt

SQL — quý hiện tại:

```
SELECT * FROM "BangDeXuat"
WHERE "createdAt" >= DATE_TRUNC('quarter', CURRENT_DATE)
  AND "createdAt" < DATE_TRUNC('quarter', CURRENT_DATE) + INTERVAL '3 months';
```

SQL — 12 tháng trượt:

```sql
SELECT
  TO_CHAR(DATE_TRUNC('month', "createdAt"), 'MM/YYYY') AS thang,
  COUNT(*)
FROM "BangDeXuat"
WHERE "createdAt" >= DATE_TRUNC('month', CURRENT_DATE - INTERVAL '11 months')
GROUP BY DATE_TRUNC('month', "createdAt")
ORDER BY 1;
```

O.21 — Skip locked: queue worker pattern

Nghiệp vụ: Background worker xử lý job recalc, không lock toàn bảng.

SQL `FOR UPDATE SKIP LOCKED`:

```sql
BEGIN;
SELECT * FROM "RecalcQueue"
WHERE status = 'PENDING'
ORDER BY created_at LIMIT 10
FOR UPDATE SKIP LOCKED;

UPDATE "RecalcQueue" SET status = 'DONE' WHERE id = ANY($1);
COMMIT;
```

`SKIP LOCKED` cho phép nhiều worker chạy song song không tranh chấp 10 row giống nhau.

Trong project: chưa có queue worker. Khi scale sẽ chuyển BullMQ + Redis hoặc Postgres queue + skip locked.

O.22 — Optimistic locking chống lost update

Vấn đề: Hai user mở form sửa cùng quân nhân, save gần nhau → user save sau đè user save trước.

Cơ chế: Thêm cột `version: Int @default(0)`.

```typescript
const personnel = await prisma.quanNhan.findUnique({ where: { id } });
// FE submit kèm personnel.version

const updated = await prisma.quanNhan.updateMany({
  where: { id, version: clientVersion },
  data: { ...newData, version: clientVersion + 1 },
});
if (updated.count === 0) {
  throw new ConflictError('Bản ghi vừa được người khác sửa, vui lòng tải lại.');
}
```

Trong project: chưa có cột `version` riêng. Em dựa vào `updatedAt` cho 1 vài endpoint quan trọng — đề xuất cải tiến.

O.23 — Pessimistic lock với SELECT FOR UPDATE

Khi cần: Ngăn 2 transaction đọc cùng row rồi update đè.

```
await prisma.$transaction(async tx => {
  await tx.$queryRaw`SELECT * FROM "TaiKhoan" WHERE id = ${id} FOR UPDATE`;
  await tx.taiKhoan.update({ where: { id }, data: { ... } });
});
```

Trade-off: Block transaction khác → dùng cẩn thận, transaction phải nhanh (< 100 ms).

O.24 — Advisory lock cho job singleton

SQL:

```
SELECT pg_try_advisory_lock(12345);
-- ... chạy job ...
SELECT pg_advisory_unlock(12345);
```

Trong project: Em dùng flag `isRunning` trong process — đủ cho single instance. Khi cluster phải chuyển advisory lock.

O.25 — EXPLAIN ANALYZE để debug query chậm

Cách dùng:

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)
SELECT * FROM "DanhHieuHangNam" WHERE quan_nhan_id = 'cl...' AND nam BETWEEN 2020 AND
        2024;
```

Đọc output: - `Seq Scan` → quét toàn bảng → cần index. - `Index Scan` → dùng index → tốt. - `Bitmap Heap Scan` → dùng index nhưng nhiều row → trung bình. - `cost=...` ước tính, `actual time=...` thực tế. - Estimate sai nhiều → chạy `ANALYZE <table>` cập nhật statistics.

Trong project: chưa có endpoint chậm critical. Bật `log: ['query']` Prisma để xem SQL sinh ra, EXPLAIN ANALYZE thủ công nếu cần.

O.26 — Index types: B-tree, GIN, GiST, partial, covering

Loại	Khi dùng	Ví dụ
B-tree (mặc định)	Equality, range, ORDER BY	`@@index([action, createdAt])` SystemLog
GIN	JSONB, array, full-text	(đề xuất) cho `payload` JSONB
GiST	Range, geometry	(chưa dùng)
Partial	Index chỉ cho subset row	`WHERE status = 'PENDING'`
Covering (`INCLUDE`)	Đọc data từ index, không heap fetch	`INCLUDE (ho_ten)`

Partial index:

```
CREATE INDEX idx_pending_proposals ON "BangDeXuat" ("createdAt")
WHERE status = 'PENDING';
```

Index nhỏ hơn, query “lấy đề xuất pending mới” cực nhanh.

O.27 — Khi nào nên thêm index? Đánh đổi gì?

Thêm khi: - Query thường xuyên (>100 lần/ngày). - Cột dùng trong WHERE / JOIN / ORDER BY. - Selectivity cao (vd: `status` 4 giá trị → index không hiệu quả; CCCD unique → rất hiệu quả).

Đánh đổi: - Index tốn disk (~10-30 % size table). - Mỗi INSERT/UPDATE/DELETE phải update index → ghi chậm. - Quá nhiều → planner chọn nhầm.

Quy tắc: Bắt đầu 0 index, thêm theo `pg_stat_user_indexes` cho thấy index không dùng → drop; query slow → thêm.

O.28 — VACUUM, ANALYZE, REINDEX — khi nào?

- **`ANALYZE`:** Cập nhật statistics planner. Sau bulk INSERT/UPDATE > 10 % size.
- **`VACUUM`:** Giải phóng space của row đã DELETE/UPDATE. Autovacuum chạy tự động.
- **`VACUUM FULL`:** Compact table — block toàn table, dùng khi disk phình.
- **`REINDEX`:** Rebuild index khi bloat.

Trong project: dựa vào autovacuum mặc định, không can thiệp.

O.29 — Connection pooling tuning

Mặc định Prisma: `connection_limit = num_cpu * 2 + 1`.

Thay đổi qua URL:

```
postgresql://user:pass@host:5432/db?connection_limit=20&pool_timeout=10
```

Khi pool exhaust: Request mới phải đợi → log error “Timed out fetching a new connection”. Cải thiện: 1. Tăng `connection_limit` (không quá `max_connections` Postgres). 2. Dùng PgBouncer transaction pooler. 3. Giảm transaction time.

O.30 — Transaction isolation levels

Level	Dirty read	Non-repeatable read	Phantom read	Mặc định Postgres
READ UNCOMMITTED	Có	Có	Có	Không hỗ trợ
READ COMMITTED	Không	Có	Có	**Có (mặc định)**
REPEATABLE READ	Không	Không	Postgres: không	Có
SERIALIZABLE	Không	Không	Không	Có (chậm nhất)

Trong project: READ COMMITTED đủ cho > 95 % case. Chỉ dùng SERIALIZABLE khi rule cần nhất quán cao (vd: gắn số quyết định không trùng):

```
await prisma.$transaction(
  async tx => { ... },
  { isolationLevel: Prisma.TransactionIsolationLevel.Serializable }
);
```

O.31 — Tại sao CUID thay vì UUID hay BIGSERIAL?

- **BIGSERIAL:** dễ đoán → enumerate attack. Defense in depth → không nên expose ID liên tục.
- **UUID v4:** 128-bit random, an toàn. Nhược: không sortable theo thời gian.
- **CUID:** 25 ký tự, sortable theo thời gian (prefix timestamp), URL-safe, collision-resistant. Phù hợp web app.

Schema hiện tại:

```
id String @id @default(cuid()) @db.VarChar(30)
```

CUID2 (mới hơn) còn an toàn hơn: thêm entropy, không leak server identity. Có thể migrate sau.

O.32 — Prisma sinh SQL kém hiệu quả — fix thế nào?

Triệu chứng: Query Prisma chậm, EXPLAIN cho thấy SQL Prisma sinh ra dùng nhiều JOIN không cần.

Cách debug: Bật `log: ['query']` → copy SQL → EXPLAIN ANALYZE trong psql.

Workaround: - Đơn giản hoá `include`. - Tách thành 2 query với `Promise.all` thay vì 1 mega-query. - Fallback `$queryRaw` cho query phức tạp.

```
// Chậm — 1 query 4 cấp
const data = await prisma.quanNhan.findMany({
  include: {
    DanhHieuHangNam: { include: { FileQuyetDinh: true } },
    LichSuChucVu: { include: { ChucVu: true } },
  },
});

// Nhanh — 3 query song song + ghép trong code
const [quanNhans, danhHieus, lichSus] = await Promise.all([
  prisma.quanNhan.findMany(),
  prisma.danhHieuHangNam.findMany({ include: { FileQuyetDinh: true } }),
  prisma.lichSuChucVu.findMany({ include: { ChucVu: true } }),
]);
```

O.33 — Migration với data transformation phức tạp (zero downtime)

Tình huống: Đổi `cap_bac String` thành `cap_bac_id String FK` referencing bảng mới `CapBac`.

Quy trình: 1. **Migration 1:** Tạo bảng `CapBac` + cột `cap_bac_id` mới (nullable). Schema cũ giữ nguyên. 2. **Backfill:** Script SQL `INSERT INTO CapBac` các giá trị unique từ `QuanNhan.cap_bac`. Update `cap_bac_id` based trên text. 3. **Đổi code:** App ghi cả 2 cột (dual-write). Đọc cột mới, fallback cột cũ. 4. **Migration 2:** Drop cột cũ. Đổi code đọc/ghi chỉ cột mới.

Trong project: Em có scenario tương tự khi rename `so_quyet_dinh` thành hard FK với `FileQuyetDinh` (commit `29f741f`) — dùng raw SQL `ALTER TABLE ... RENAME COLUMN` để giữ data, sau đó `db push`.

O.34 — Backup chiến lược chuyên sâu

Cấp	Công cụ	Đặc điểm
Logical	`pg_dump`	SQL text, dễ restore từng phần, chậm cho DB lớn. Em dùng.
Physical	`pg_basebackup`	Snapshot file system, nhanh, restore toàn bộ.
WAL archiving	`archive_command`	Continuous backup → point-in-time recovery (PITR).

Strategy đề xuất khi scale: - pg_basebackup hằng tuần. - WAL archive liên tục. - pg_dump hằng ngày (logical, dễ migrate). - Test restore mỗi tháng.

O.35 — Prisma version 6 breaking change — em xử lý sao?

- Prisma đánh dấu rõ breaking change trong CHANGELOG.
- Em pin version trong `package.json` (`"prisma": "5.10.2"`, không dùng `^`).
- Khi upgrade: đọc migration guide, chạy regression test (870 ca), sửa breaking nếu có.
- Có thể giữ version cũ vài năm nếu Prisma vẫn hỗ trợ.

Phụ chú: 20 câu hỏi tủ thường gặp khi bảo vệ web

1. Demo nhanh chức năng quan trọng nhất.
2. Vẽ tay sơ đồ kiến trúc trên giấy.
3. Giải thích flow đăng nhập từ click submit đến cookie set.
4. Tại sao chia thành nhiều bảng thay vì 1 bảng to?
5. Foreign key onDelete Cascade vs Restrict — khi nào dùng cái nào?
6. Tại sao dùng cuid thay vì uuid hoặc auto-increment?
7. Migration đã chạy 50 lần — làm sao quản lý version?
8. Test coverage 85 % nghĩa là gì? 15 % còn lại là gì?
9. Nếu bỏ Prisma, cần thay đổi bao nhiêu file?
10. Prisma có hỗ trợ MySQL không? Migrate cần làm gì?
11. JWT secret leak — quy trình rotation thế nào?
12. Chứng thực hai yếu tố (2FA) — em có nghĩ đến không?
13. Vi phạm GDPR / Luật bảo vệ dữ liệu cá nhân — có quan tâm không?
14. Logging có bao gồm password không? (Trả lời: KHÔNG)
15. Nếu admin xoá nhầm 100 quân nhân — recovery thế nào?
16. Có rate-limit cho download file không?
17. CORS đặt `*` không? (Trả lời: KHÔNG, whitelist 1 origin)
18. Đo performance bằng tool gì? (Có thể dùng `autocannon`, `wrk` cho load test)
19. CI/CD có không? (Hiện chưa, đề xuất GitHub Actions tương lai)
20. Nếu được làm lại, em sẽ thay đổi gì? (Câu hỏi đánh giá tự phản tỉnh — có sẵn câu trả lời)

Câu trả lời cho câu 20 — “Em sẽ làm khác gì?”

“Có ba điều em sẽ làm khác. **Một**, em sẽ áp dụng Repository pattern ngay từ đầu thay vì tách sau khi service đã lớn — sẽ tiết kiệm 1 tuần refactor. **Hai**, em sẽ viết test cho Controller layer ngay từ đầu, không chỉ Service — coverage Controller hiện chỉ 60 %. **Ba**, em sẽ dùng `tRPC` hoặc shared schema package để tránh duplicate Joi/Zod giữa BE và FE. Tuy nhiên những điểm này không phải critical, em vẫn hài lòng với kiến trúc tổng thể đã chọn.”

Chúc bạn bảo vệ thành công. Hệ thống đã đầy đủ tính năng, có số đo định lượng rõ ràng, có audit log đầy đủ, có 870 test pass — đều là vũ khí mạnh khi hội đồng truy vấn. Khi đứng trước hội đồng, hít sâu, nói chậm, mắt nhìn vào người hỏi và đừng quên: **mọi thứ trong đồ án này em đã sống với 6 tháng — em là người hiểu nó nhất phòng**.